

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 12, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	8
2.3	Rust Implementation	9
3	Derivatives and Partial Derivatives	11
3.1	Deciding Membership	11
3.2	Brzozowski Derivatives	12
3.3	Antimirov Partial Derivatives	15
3.4	Matcher Selection	17
4	Matching versus Parsing	19
4.1	The Matching Problem	19
4.2	Parse Trees as Matching Evidence	21
4.3	Disambiguation Policies	24
4.3.1	Greedy Leftmost Matching	25
4.3.2	The Infinite Empty-Match Chain Problem	26
4.3.3	POSIX Longest-Leftmost Matching	27
5	Derivative-Based Parser	32
5.1	Core Algorithm	32
5.2	Correctness Properties	34
5.3	Bit-Coded Optimization (Theory)	34
5.4	Simplification Rules	37
5.5	Rust Implementation: The <code>deriv_std</code> Parser	37
5.5.1	<code>ParseTree</code> and <code>flatten</code>	38
5.5.2	<code>mk_eps</code>	38
5.5.3	<code>inject</code>	38
5.5.4	<code>parse_recursive</code> vs. <code>parse_loop</code>	38
5.6	Rust Implementation: The Bit-Coded Version	39
5.6.1	<code>ARegex</code> – Bit-Annotated Regex	39
5.6.2	<code>internalize/fuse</code>	39

5.6.3	<code>deriv_bc</code>	39
5.6.4	<code>mk_eps_bc</code>	40
5.6.5	<code>decode</code>	40
5.7	Worked Example: <code>paper_r1</code> End to End	40
6	Partial-Derivative-Based Parser	47
6.1	Extending Derivatives to Partial Derivatives	47
6.2	Bit-Coded Variant	48
6.3	Rust Implementation: The <code>pderiv_bc</code> Parser	49
6.4	Rust Implementation: The <code>pderiv_std</code> Parser	50
6.5	Worked Example: <code>paper_r1</code> End to End	50
7	Frontend: Parsing External Regular Expression Syntax	57
7.1	Two ASTs, One Lowering Step	57
7.2	Parsing Surface Syntax	58
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	58
7.4	Substring Search	60
8	Evaluation	61
8.1	Compared Engines	61
8.1.1	Rust’s <code>regex</code> Crate	61
8.1.2	<code>RE2</code>	62
8.2	Testing Strategy	62
8.3	Dataset Preparation	62
8.4	Correctness Verification	64
8.4.1	Paper Examples Reproduced	64
8.4.2	Cross-Implementation Equivalence	64
8.5	Performance Analysis	65
8.5.1	Stack Depth: Recursive vs. Iterative Construction	65
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	65
8.5.3	Derivative-Based vs. Partial-Derivative-Based	65
8.5.4	Pathological Inputs	65
8.5.5	Benchmark Results	66
8.6	Comparison with Existing Engines	66
8.6.1	Performance	66
8.6.2	Standardized Benchmarking via <code>rebar</code>	66
9	Discussion	67
9.1	Discussion of Evaluation Results	67
9.2	Rust versus Haskell	67
9.2.1	Language Characteristics of the Reference Implementation	67
9.2.2	What the Reference Provides vs. What It Leaves Open	68
9.2.3	Translation Strategy	68
9.2.4	Translation Insights	68
9.3	Mismatch Analysis via Derivative Tracing	68
9.3.1	The Gap	68
9.3.2	Deriving Diagnostics for Free from the Algorithm Itself	68
9.3.3	Design	68
9.3.4	Scope and Limits	69

9.4	Optimization Challenges and Insights	69
9.5	Future Work	69
9.5.1	Paper-Suggested Extensions	69
9.5.2	Further Performance Optimizations	69
9.5.3	A Full Mismatch Diagnostics System	69
10	Related Work and Conclusion	70
A	Code Listings	73
B	Full Benchmark Results	74
C	Test Case Catalogue	75
D	CLI and ENV Reference	76

Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching: given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches the input string "aba", as it can be decomposed into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce that input.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password, URL, or configuration value. Other applications require more: a lexer must determine how an input decomposes into tokens, while capturing groups, schema validation, and information extraction require identifying which parts of the input correspond to different parts of the expression. In these settings, recognition alone is insufficient: a matcher must also recover the *structure* of a match, turning a membership problem into a parsing problem.

Efficiently solving these problems remains an active research area: the choice of algorithm influences performance, memory consumption, and security, and, once matching becomes parsing, the choice of how a match's structure is recovered influences correctness as well.

Existing approaches include backtracking, finite-automata-based techniques such as non-deterministic (NFA) and deterministic (DFA) automata. Each approach provides different trade-offs between implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely adopted, but may exhibit exponential running time on carefully constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require costly preprocessing and can suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression itself as input symbols are consumed. Each derivative represents the language that remains to be matched after a particular input symbol has been consumed. By maintaining a direct correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less commonly used in practical regular expression tooling than backtracking and automata-based approaches. Investigating how these techniques behave in a systems implementation, and whether they can support practical matching, parsing, and diagnostic tools, therefore provides the motivation for this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based regular expression matching has been studied extensively from a theoretical perspective. Brzozowski introduced regular expression derivatives as an algebraic characterization of regular language membership [Brzozowski, 1964], while Antimirov introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996]. Building on these ideas, Sulzmann and Lu developed a derivative-based algorithm for POSIX-correct regular expression parsing and provided a Haskell reference implementation [Sulzmann and Lu, 2014].

Sulzmann and Lu's work therefore provides a formally developed basis for structured regular expression matching, but leaves several practical questions open. In particular, the behaviour of the construction in a systems programming language can be investigated with respect to execution time, memory consumption, and other implementation-level resource usage. Rust provides an appropriate setting for such an investigation through its ownership model and fine-grained control over data representation and memory allocation.

A further question is whether the construction can be extended to use Antimirov's partial derivatives as an alternative representation of residual expressions, and how such an implementation compares with the original derivative-based construction. The bit-coded representation proposed by Sulzmann and Lu also provides an opportunity to investigate the practical effect of reducing the memory required to represent derivative expressions.

The central problem addressed by this thesis is therefore to investigate the practical behaviour of derivative-based approaches to POSIX parsing: first, by implementing Sulzmann and Lu's construction in Rust and evaluating its performance and resource usage; second, by investigating an alternative construction based on partial derivatives; and third, by comparing the resulting implementations with established regular expression engines. In addition, the thesis investigates whether the intermediate derivative computations can be exposed to provide useful explanations of successful and unsuccessful matches.

1.3 Contributions

This thesis makes the following contributions:

- **A Rust implementation of Sulzmann and Lu's derivative-based POSIX parser.** The implementation includes both a direct two-pass construction following the original algorithm and a bit-coded single-pass optimization. It additionally explores Antimirov's partial derivatives as an alternative representation, resulting in four parser variants: derivative-based and partial-derivative-based implementations, each in direct and bit-coded forms.

- **An empirical evaluation of the parser variants.** The four variants are evaluated against one another using synthetic benchmarks and a corpus of real-world regular expressions based on the datasets used by Mamouras et al. [Mamouras et al., 2024]. Their performance is also compared with established regular expression engines, including Rust’s `regex` crate and Google’s RE2.
- **A derivative-tracing diagnostic system.** The implementation exposes the intermediate derivative computations performed during matching, providing a basis for explaining successful and unsuccessful matches and for investigating the behaviour of the parser.

1.4 Thesis Outline

The remainder of this thesis is structured as follows.

Chapter 2 introduces the syntax and semantics of regular expressions, together with the Rust representation of their syntax used throughout the rest of this work.

Building on this representation, Chapter 3 introduces Brzozowski’s derivatives and Antimirov’s partial derivatives as algorithms for deciding language membership, along with their Rust implementation.

Before extending these algorithms to parsing, Chapter 4 distinguishes matching from parsing and introduces the disambiguation problem that arises once a parse tree, not just a Boolean, is required.

Then Chapter 5 presents Sulzmann and Lu’s derivative-based POSIX parser [Sulzmann and Lu, 2014], which serves as the algorithmic basis of this thesis, together with its Rust implementation in both a direct and a bit-coded form.

Chapter 6 extends this construction from Brzozowski derivatives to Antimirov’s partial derivatives, again in both its direct and bit-coded variants, resulting in a Greedy parser.

The translation of the input regular expression into the internal representation used by the parsers is described in Chapter 7, presenting the front-end of the implementation.

With all four parsers variants in place, Chapter 8 evaluates their correctness and performance against each other, on both synthetic benchmarks and a corpus of real-world regular expressions, and positions this evaluation relative to established regular expression engines.

Chapter 9 discusses and interprets the evaluation results, situates the work relative to related approaches, and reflects on the translation of the underlying algorithm from Haskell to Rust. It also discusses the derivative-tracing system, its usefulness and limitations.

Finally, Chapter 10 concludes the thesis by summarizing the main findings and contributions and relating them to the research questions introduced in this chapter. It also discusses directions for future work, including potential extensions and improvements to the implementations and diagnostic tooling.

Regular Expressions

2.1 Syntax

Over an alphabet Σ , regular expressions are constructed from literal symbols, the expressions ε and $\emptyset$, and the three operators alternation, concatenation, and Kleene star. The syntax is given by the following grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma)$$

Here x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word at all. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and repetition (Kleene star), respectively. Concatenation is frequently left implicit, writing rs for $r \cdot s$.

Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. Thus, for example, ab^*c is parsed as $(a \cdot (b^*)) \cdot c$, and $a + bc^*$ is parsed as $a + (b \cdot (c^*))$, not $(a + b) \cdot (c^*)$.

Both $+$ and $\cdot$ are associative as operations on languages – $L((r + s) + t) = L(r + (s + t))$ and $L((r \cdot s) \cdot t) = L(r \cdot (s \cdot t))$ follow directly from the semantics of Section 2.2 – so which of $r + s + t$'s two possible groupings the grammar above picks out never changes the language denoted. It does, however, fix a specific *syntax tree*, and the algorithms of later chapters operate on that tree rather than on $L(r)$ directly, so the convention needs to be fixed unambiguously: both operators associate to the *left*, so $r + s + t$ means $(r + s) + t$ and rst means $(rs)t$. Section 2.3 makes this concrete for the running example r_{01} , whose concatenation $(0+1)^* \cdot 0 \cdot 1$ left-associates into the tree $((0+1)^* \cdot 0) \cdot 1$ – the exact grouping Section 3.2 and Section 3.3 derive against, character by character.

The grammar above deliberately has no one-or-more, optional, or bounded repetition operator (r^+ , $r?$, $r^{\{m,n\}}$): every algorithm in this thesis is defined over exactly the six constructs above, and nothing else. This is not a limitation on what can be matched – each of those convenience operators is equivalent to some expression already expressible here ($r^+ = r \cdot r^*$, for instance) – only on what the core language needs to define recursion over directly. Chapter 7 (Section 7.3) gives the complete table of such desugarings, applied by the frontend before any of the matchers or parsers implemented ever see the result.

As a running example, consider the problem of recognizing binary strings that end in "01", a small but non-trivial pattern that uses all three operators in the grammar above. Over $\Sigma = 0, 1$, this pattern is expressed as

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1.$$

Reading this expression according to the grammar, $(0 + 1)$ is an alternation of two literals, $(0 + 1)^*$ applies Kleene star to that alternation, and the result is concatenated with the literals 0 and 1 in sequence.

2.2 Semantics

The syntax above describes the *shape* of an expression; the semantics assigns each expression the *language* it denotes. For a regular expression r , write $L(r)$ for the set of words it matches. The semantics is defined by structural recursion on r :

$$\begin{aligned} L(x) &= \{x\} \\ L(\varepsilon) &= \{\varepsilon\} \\ L(\emptyset) &= \{\} \\ L(r + s) &= L(r) \cup L(s) \\ L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\} \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid w_1, \dots, w_n \in L(r), n \geq 1\} \end{aligned}$$

The following examples illustrate the distinction between an expression and the language it denotes. Different expressions may describe related or even identical sets of words, while the semantics assigns a precise language to each expression.

Expression	Language
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

The distinction between an expression and its denoted language is important for the algorithms developed later. Derivative operations are defined syntactically over regular expressions, while their correctness is understood in terms of the languages those expressions denote.

A word w *matches* r exactly when $w \in L(r)$. This is the membership problem.

Unwinding the semantics for the running example, $L(0 + 1) = 0, 1$, so $L((0 + 1)^*)$ is the set of all finite binary strings, including the empty string. Concatenating with $L(0) = 0$ and then $L(1) = 1$ restricts this to

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\},$$

i.e. exactly the binary strings ending in "01" like "01", "001", "1101", and so on. In particular, the empty string is not matched, since every word in $L(r_{01})$ contains the final literal 0 followed by the literal 1.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum, mirroring each syntactic construct one to one. Rust enums define a type by enumerating its possible variants, and are commonly used together with pattern matching to distinguish between those variants [The Rust Project Developers, c,h].

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

In this representation, each production of the grammar corresponds to one enum variant. For example, `Phi` represents $\emptyset$, `Eps` represents ε , `Lit` represents a literal symbol, and the remaining variants represent the three recursive constructs.

For example, the expression

$$(0 + 1)^*01$$

is represented structurally by an expression tree whose root is a concatenation, with $(0+1)^*$ and 01 as its subexpressions. More explicitly, its structure can be viewed schematically as

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1).$$

Thus, the Rust value is not merely a string representation of the regular expression; it is a tree that directly represents its syntactic structure. This structural representation allows operations to be defined recursively over the expression.

Rust's `match` expressions are checked for exhaustiveness by the compiler, meaning that every possible pattern must be covered either explicitly or through a wildcard pattern [The Rust Project Developers, d,i]. This mirrors the structural guarantee provided by the inductive definition of r in Section 2.1: a function defined by cases on r must account for every possible form of r .

The derived traits support operations used throughout the implementation. The `derive` attribute generates implementations of supported traits for the annotated type [The Rust Project Developers, b]. In particular, `Debug` facilitates inspection of expressions, `Clone` allows expressions to be duplicated when required, and `PartialEq`, `Eq`, and `Hash` allow

expressions to be compared and used as keys in hash-based collections. Rust’s `HashMap`, for example, requires its key type to implement both `Eq` and `Hash` [The Rust Project Developers, g,f]. The variants `Phi` and `Eps` carry no data, corresponding directly to $\emptyset$ and ε , while `Lit` stores the literal symbol x . In the implementation, literal symbols are represented using Rust’s `char` type and therefore correspond to Unicode scalar values [The Rust Project Developers, e].

The recursive constructs, `Seq`, `Alt`, and `Star` require explicit indirection in Rust. A recursive type must have a finite size, so a definition such as `Seq(Regex, Regex)` would not have a finite size: each `Regex` would contain another `Regex`, and so on. `Box<Regex>` provides the necessary indirection by storing the recursive value separately and keeping a fixed-size pointer-like value within the enum [The Rust Project Developers, a,j].

The use of `Box` is therefore one of the practical differences between this Rust representation and the Haskell reference implementation. It introduces explicit indirection and heap-based storage for recursive subexpressions, aspects that are relevant when considering the memory behaviour of the implementation. These implementation differences are revisited more generally in Section 9.2.

With this representation established, operations on regular expressions can be defined structurally over the expression tree.

Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions. The semantics provides a declarative definition of membership: a word w matches an expression r exactly when $w \in L(r)$. This chapter turns that definition into executable algorithms for deciding membership.

Three approaches are considered. The first is a direct recursive matcher that follows the structure of the semantic definition. The second uses Brzozowski derivatives to transform an expression as input symbols are consumed. The third uses Antimirov’s partial derivatives, representing the possible residual expressions as a collection rather than combining them into a single expression. All three are implemented in Rust with the common interface `fn(&str, &Regex) -> bool`.

The three approaches decide the same membership problem, but they differ in how they represent and explore the remaining matching problem. The derivative-based approaches are particularly important for this thesis: the derivative operation introduced here forms the basis of the structured parsing algorithms developed in Chapters 5 and 6.

3.1 Deciding Membership

The semantics of Section 2.2 defines membership declaratively: a word w matches a regular expression r exactly when $w \in L(r)$. A direct way to decide membership is therefore to interpret these semantic definitions as recursive matching rules.

Let w be represented as an indexable sequence of characters, and let $w[i : j]$ denote the substring beginning at position i and ending immediately before position j . A recursive procedure `match(w, i, j, r)` can then determine whether $w[i : j] \in L(r)$ by following the structure of r :

- ε matches $w[i : j]$ iff $i = j$.
- A literal x matches $w[i : j]$ iff $j = i + 1$ and the character at position i is x .
- $r + s$ matches $w[i : j]$ iff r matches $w[i : j]$ or s does.
- $r \cdot s$ matches $w[i : j]$ iff there is some split point $k \in [i, j]$ such that r matches $w[i : k]$ and s matches $w[k : j]$.
- r^* matches $w[i : j]$ iff $i = j$, or there is some $k \in (i, j]$ such that r matches $w[i : k]$ and r^* matches $w[k : j]$.

Membership of the whole word w corresponds to `match(w, 0, |w|, r)`. This procedure is

correct by construction: every case reproduces the corresponding case of $L(r)$ directly, so nothing beyond the semantic definition itself needs to be proved.

It is also, in general, an expensive procedure. Concatenation and star both search over every split point in the current range rather than committing to one, and nothing remembers a range once it has already been explored while resolving a different call. Take $r = a \cdot a \cdot a$ against $w = \text{aaa}$. Matching $w[0 : 3]$ against r tries every split point $k \in \{0, 1, 2, 3\}$ for the outer concatenation, even though only $k = 1$ can possibly succeed, since the left operand is a single literal and can only match a range of length exactly 1. The right operand at that point, $w[1 : 3]$ against $a \cdot a$, repeats the same search one level down. For a chain of n literals against an input of length n , the number of ranges examined this way grows exponentially in n rather than linearly, since each concatenation node re-explores the full range handed to it regardless of what a sibling call already tried.

`src/matchers/match_naive.rs` implements this procedure directly on a `Vec<char>` and a pair of `usize` bounds. Despite its cost, it remains useful: because it transcribes $L(r)$'s definition without any intervening transformation, it serves as an independent reference that the more efficient matchers introduced below can be checked against. The remainder of this chapter develops two such matchers, both based on a different idea: instead of searching for a decomposition of w against a fixed r , *transform* r itself as each character of w is consumed, so that membership of the remaining suffix is asked against a smaller, adjusted expression rather than re-asked against the same r every time.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \backslash x$, is the expression describing exactly what remains of r 's language once a leading x has been removed [Brzowski, 1964]:

$$L(r \backslash x) = \{ w \mid xw \in L(r) \}.$$

Deciding whether a word $x_1 x_2 \cdots x_n$ is in $L(r)$ then reduces to computing the chain $r \backslash x_1 \backslash x_2 \backslash \cdots \backslash x_n$ and checking whether the final expression accepts the empty word, i.e. whether it is *nullable*. Both nullability and the derivative itself are defined by structural recursion directly on the syntax of r , with no reference back to $L(r)$ needed once the base cases are fixed:

$$\begin{array}{lll} n(\emptyset) = \text{false} & n(\varepsilon) = \text{true} & n(x) = \text{false} \\ n(r + s) = n(r) \vee n(s) & n(r \cdot s) = n(r) \wedge n(s) & n(r^*) = \text{true} \end{array}$$

$$\begin{array}{lll} \emptyset \backslash x = \emptyset & \varepsilon \backslash x = \emptyset & y \backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} \\ (r + s) \backslash x = r \backslash x + s \backslash x & r^* \backslash x = (r \backslash x) \cdot r^* & \\ (r \cdot s) \backslash x = \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) \\ (r \backslash x) \cdot s & \text{otherwise} \end{cases} & & \end{array}$$

The only case that needs justification beyond direct case analysis on $L(r \cdot s)$ is concatenation: if r is nullable, a word starting with x can either continue matching r (so the derivative goes into r while s is left untouched) or arise from r already being satisfied by ε , with x belonging to s instead. Hence the extra $s \setminus x$ summand appears exactly when $n(r)$ holds. If r is not nullable, this second possibility cannot arise, since $\varepsilon \notin L(r)$ rules out r having already been satisfied before any input was consumed.

Worked example: single-symbol matching. Consider x^* against the word **xx**. Writing $r \xrightarrow{x} r \setminus x$ for one derivative step,

$$x^* \xrightarrow{x} (x \setminus x) \cdot x^* = \varepsilon \cdot x^* \xrightarrow{x} (\varepsilon \setminus x) \cdot x^* + x^* \setminus x = \emptyset \cdot x^* + (x \setminus x) \cdot x^* = \emptyset \cdot x^* + \varepsilon \cdot x^*.$$

Checking nullability of the final expression by structural recursion,

$$\begin{aligned} n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) \\ &= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) = \text{true}, \end{aligned}$$

so x^* matches **xx**, correctly. This short trace makes two things visible. First, the expression has grown from one node to five across two steps, purely from the $+$ the concatenation rule introduces at each nullable-prefix step; nothing above bounds this growth, and a third derivative step would nest a further $+$ inside each existing summand. Second, the identities listed below collapse the final expression straight back down to $\varepsilon \cdot x^*$, which is exactly the expression reached after only *one* derivative step, not two. Without them, a derivative sequence accumulates $\emptyset$'s and ε 's that never get removed; with them, the sequence for x^* never grows past a single node.

Four identities do this collapsing, each a direct consequence of the semantics of Section 2.2:

- $\emptyset \cdot r = \emptyset$
- $\varepsilon \cdot r = r$
- $\emptyset + r = r$
- $r + r = r$ (idempotent alternation)

`src/regex/simplify.rs` implements exactly these four rules, applied to every subexpression; `match_deriv` (`src/matchers/match_deriv.rs`) calls it after every derivative step, which is why neither worked example in this section ever shows an expression larger than what is written down. Each step in the trace below is annotated with the identity it uses.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ Recall r_{01} from Section 2.1, left-associated as $((0 + 1)^* \cdot 0) \cdot 1$. Derive it against $w = 101$ one symbol at a time.

1. **Derive by 1.** The left operand $(0 + 1)^* \cdot 0$ is not nullable (it still needs its trailing 0), so the concatenation rule's second case applies:

$$r_{01} \setminus 1 = (((0 + 1)^* \cdot 0) \setminus 1) \cdot 1.$$

The inner star $(0 + 1)^*$ is nullable, so its own derivative needs the concatenation rule's first case too:

$$(0+1)^*\backslash 1 = ((0\backslash 1)+(1\backslash 1))\cdot(0+1)^* = (\emptyset+\varepsilon)\cdot(0+1)^* = (0+1)^* \quad (\text{by } \emptyset + r = r, \varepsilon \cdot r = r).$$

Substituting back, since $(0 + 1)^*$ is nullable:

$$((0+1)^*\cdot 0)\backslash 1 = ((0+1)^*\backslash 1)\cdot 0 + (0\backslash 1) = (0+1)^*\cdot 0 + \emptyset = (0+1)^*\cdot 0 \quad (\text{by } \emptyset + r = r).$$

So

$$r_{01}\backslash 1 = ((0 + 1)^* \cdot 0) \cdot 1 = r_{01}.$$

A 1 that is not yet part of the final 01 loops back to the start, since $(0 + 1)^*$ can absorb it.

2. **Derive by 0.** The same computation, with 0 and 1 exchanged: $(0 + 1)^*\backslash 0$ again simplifies to $(0 + 1)^*$, but this time $0\backslash 0 = \varepsilon$ rather than $\emptyset$, so

$$((0 + 1)^* \cdot 0)\backslash 0 = (0 + 1)^* \cdot 0 + \varepsilon$$

does *not* collapse further and neither summand is $\emptyset$, and the two are not identical. This gives

$$r_{01}\backslash 1\backslash 0 = ((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1.$$

It reads exactly as it should: after "10", either the trailing 01 is still owed in full (the $(0 + 1)^* \cdot 0$ summand), or the just-consumed 0 was already the first half of it (the ε summand); both possibilities stay open until the next symbol decides between them.

3. **Derive by 1.** $(0 + 1)^* \cdot 0 + \varepsilon$ is nullable (via its ε summand), so the concatenation rule's first case applies:

$$(((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1)\backslash 1 = (((0 + 1)^* \cdot 0 + \varepsilon)\backslash 1) \cdot 1 + (1\backslash 1).$$

Working out the inner derivative, reusing step 1's result that $((0+1)^*\cdot 0)\backslash 1$ simplifies to $(0 + 1)^* \cdot 0$:

$$((0+1)^*\cdot 0+\varepsilon)\backslash 1 = (((0+1)^*\cdot 0)\backslash 1)+(\varepsilon\backslash 1) = (0+1)^*\cdot 0 + \emptyset = (0+1)^*\cdot 0 \quad (\text{by } \emptyset + r = r).$$

So

$$r_{01}\backslash 1\backslash 0\backslash 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon.$$

$r_{01} + \varepsilon$ is nullable, since $n(r_{01} + \varepsilon) = n(r_{01}) \vee n(\varepsilon) = \text{false} \vee \text{true} = \text{true}$, so $101 \in L(r_{01})$, correctly: 101 ends in 01. The final ε summand is exactly what witnesses this: it says the match was already complete the moment the third symbol was read, which is precisely what makes the whole expression nullable.

Now for only the word, $w = 10$. It shares its first two symbols with 101, so deriving r_{01} by 1 then by 0 produces the same intermediate expression as step 2 above, $((0+1)^*\cdot 0+\varepsilon)\cdot 1$, but since 10 has only two symbols, this *is* the final result for this word, not an intermediate one. Checking its nullability directly: the expression is *not* nullable, since neither summand of $(0 + 1) \cdot 0 + \varepsilon$ makes the whole concatenation with 1 nullable: $(0 + 1)^* \cdot 0$ is not nullable, and $\varepsilon \cdot 1$ still needs the literal 1. So $10 \notin L(r_{01})$, correctly: 10 ends in 0, not 01.

`src/regex/nullable.rs` and `src/regex/deriv.rs` implement $n(r)$ and $r \setminus x$ case by case, matching the `Regex` enum of Section 2.3 one variant at a time. `match_deriv` (`src/matchers/match_deriv.rs`) folds `deriv` over the input characters, applies `simplify` after every step exactly as annotated in the worked example above, and checks `nullable` on the result.

The sequence of derivatives $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \dots \xrightarrow{x_n} r_n$ can equivalently be read as tracing a path through an automaton built lazily, on demand: each r_i is a state, and $\xrightarrow{x}$ is the transition relation. At every step there is exactly one current expression, playing the role of exactly one current automaton state, however large that state's internal syntax tree happens to be a DFA-style view of the construction that Section 3.3 contrasts directly with an NFA-style one.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 keeps the residual language as a single expression, combining every alternative reached so far with $+$. Antimirov's partial derivatives take a different route to the same residual language: rather than combining the alternatives produced along the way, keep them apart as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set,

$$\begin{aligned}
pd(\emptyset, x) &= \{\} \\
pd(\varepsilon, x) &= \{\} \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) \\
pd(r^*, x) &= \{ smartC(r', r^*) \mid r' \in pd(r, x) \} \\
pd(r \cdot s, x) &= \begin{cases} \{ smartC(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) \\ \{ smartC(r', s) \mid r' \in pd(r, x) \} & \text{otherwise} \end{cases}
\end{aligned}$$

where the smart constructor $smartC(\varepsilon, s) = s$ and $smartC(r', s) = r' \cdot s$ otherwise avoids introducing a spurious $\varepsilon \cdot s$ whenever a partial derivative of r is exactly ε . This keeps every partial derivative a subexpression of r (up to the concatenations $smartC$ builds), rather than an expression that could grow without bound the way plain $r \setminus x$ can.

Worked example: single-symbol matching. Consider x^* against `xx` again, this time under pd . The base case gives $pd(x, \mathbf{x}) = \{\varepsilon\}$, and the star rule applies the smart constructor to it:

$$pd(x^*, \mathbf{x}) = \{ smartC(r', x^*) \mid r' \in pd(x, \mathbf{x}) \} = \{ smartC(\varepsilon, x^*) \} = \{x^*\},$$

since $smartC(\varepsilon, s) = s$. Deriving again by `x` repeats exactly the same computation, because the set to derive is still $\{x^*\}$: $pd(x^*, \mathbf{x}, \mathbf{x}) = \{x^*\}$. This set contains x^* , which is nullable, so x^* matches `xx`, correctly, which is the same conclusion as the derivative trace,

reached here in two steps that never leave $\{x^*\}$. Unlike that trace, which passed through an expression carrying an $\emptyset$ and an ε neither one needed, before collapsing back down, pd never produces either kind of clutter to begin with: $pd(\emptyset, x) = \{\}$ discards empty alternatives the moment they arise, and $smartC$ discards a spurious $\varepsilon \cdot s$ the moment it would be built. No separate simplification pass is needed for this to hold, it follows directly from how pd and $smartC$ are defined, not from an additional rewriting step applied afterwards.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ Derive r_{01} against $w = 101$ again, this time tracking a set of residuals instead of one expression.

1. **Derive by 1.** As in Section 3.2, finding $pd((0 + 1)^* \cdot 0, 1)$ needs the inner star first. Since $(0 + 1)^*$ is nullable,

$$pd((0 + 1)^* \cdot 0, 1) = \{ smartC(r', 0) \mid r' \in pd((0 + 1)^*, 1) \} \cup pd(0, 1).$$

Working out $pd((0 + 1)^*, 1)$: $pd(0+1, 1) = pd(0, 1) \cup pd(1, 1) = \{\} \cup \{\varepsilon\} = \{\varepsilon\}$, so $pd((0 + 1)^*, 1) = \{ smartC(\varepsilon, (0 + 1)^*) \} = \{(0 + 1)^*\}$. Substituting back, and since $pd(0, 1) = \{\}$:

$$pd((0 + 1)^* \cdot 0, 1) = \{ smartC((0 + 1)^*, 0) \} \cup \{\} = \{(0 + 1)^* \cdot 0\}.$$

$(0 + 1)^* \cdot 0$ is not nullable, so the outer concatenation rule needs no union with $pd(1, 1)$:

$$pd(r_{01}, 1) = \{ smartC(r', 1) \mid r' \in pd((0 + 1)^* \cdot 0, 1) \} = \{(0 + 1)^* \cdot 0 \cdot 1\} = \{r_{01}\}.$$

A single residual, identical to r_{01} itself: reading a 1 that is not yet part of the final 01 loops back to the start, exactly as in the derivative trace, but here as a one-element set, rather than as one expression equal to r_{01} .

2. **Derive by 0.** The set is still $\{r_{01}\}$, so derive r_{01} by 0. The same computation as step 1, with 0 and 1 exchanged, again gives $pd((0 + 1)^*, 0) = \{(0 + 1)^*\}$, but now $pd(0, 0) = \{\varepsilon\}$ rather than $\{\}$, so

$$pd((0 + 1)^* \cdot 0, 0) = \{ smartC((0 + 1)^*, 0) \} \cup \{\varepsilon\} = \{(0 + 1)^* \cdot 0, \varepsilon\}.$$

Substituting into the outer concatenation:

$$pd(r_{01}, 1, 0) = \{ smartC(r', 1) \mid r' \in \{(0+1)^* \cdot 0, \varepsilon\} \} = \{(0+1)^* \cdot 0 \cdot 1, 1\} = \{r_{01}, 1\}.$$

This is the same information as $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$ from the derivative trace. Both say "either the trailing 01 is still owed in full, or the just-consumed 0 was already its first half", but where the derivative folds the two possibilities into one expression with an internal +, pd keeps them apart as two separate residuals, r_{01} and the literal

1, with no $+$ at all. This is the concrete sense in which pd is NFA-shaped: the two-element set is exactly the state set an NFA for r_{01} would be in after reading 10, one state per strand, rather than one DFA state encoding both strands internally.

3. **Derive by 1.** Each element of $\{r_{01}, 1\}$ is derived separately and the results unioned: $pd(r_{01}, 1) = \{r_{01}\}$, repeating step 1, and $pd(1, 1) = \{\varepsilon\}$. So

$$pd(r_{01}, 1, 0, 1) = \{r_{01}\} \cup \{\varepsilon\} = \{r_{01}, \varepsilon\}.$$

This set contains ε , so it contains a nullable residual, and $101 \in L(r_{01})$ the same conclusion as the derivative trace, reached here by checking whether any residual currently held is nullable, rather than by checking nullability of one combined expression.

Now for $w = 10$, the same way the derivative trace did: it shares its first two symbols with 101, so deriving by 1 then 0 (step 2 above) reaches the same set, $\{r_{01}, 1\}$ and since 10 has only two symbols, this *is* the final set for this word. Neither element is nullable: $n(r_{01}) = \text{false}$, and the literal 1 is nullable only once it has been derived into ε , not as itself. So $10 \notin L(r_{01})$, correctly.

The set of all partial derivatives reachable from a fixed r by derivation with respect to arbitrary words is always finite, bounded by the number of literal-symbol occurrences in r [Antimirov, 1996]. r_{01} has four such occurrences, the two inside $(0 + 1)^*$, and one each for the trailing 0 and 1, so at most four distinct partial derivatives (plus r_{01} itself) can ever appear, no matter how long a word r_{01} is derived with respect to. The worked example above already shows three of those residuals directly: every set encountered, $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$, is built entirely from r_{01} , the literal 1, and ε . Ordinary derivatives are also finite up to language equivalence. This is Brzozowski [1964]’s own finiteness result, the one that makes the DFA reading at the end of Section 3.2 possible in the first place, but only once the simplification identities of Section 3.2 are applied to identify equivalent expressions; the partial-derivative bound holds already at the syntactic level, with no such identification needed and the same point the single-symbol example above made concretely for x^* .

pd is implemented inside `match_pderiv` (`src/matchers/match_pderiv.rs`), which mirrors the worked example above directly: rather than tracking one expression, it maintains a `HashSet<Regex>` [The Rust Project Developers, f] of currently-live residuals, replacing that set at each character with the union of $pd(r', x)$ over every residual r' currently in it, and accepting once the input is exhausted if any surviving residual is nullable, which is exactly the frontier $\{r_{01}\} \rightarrow \{r_{01}, 1\} \rightarrow \{r_{01}, \varepsilon\}$ traced by hand above. This is subset simulation of an NFA whose states are subexpressions of r_{01} , run directly on the syntax rather than on a separately constructed automaton.

3.4 Matcher Selection

`match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool` despite implementing three structurally different algorithms: exhaustive search over substring splits, a single evolving DFA-style expression, and an evolving NFA-style set of expressions. This uniform interface makes the three matchers directly interchangeable: the same regular expression and input can be run through any of them, and any disagreement between their answers points to a bug in one of the three independently-written

implementations rather than to an ambiguity in what the correct answer is supposed to be. The same pattern of comparing independently-derived implementations is reused throughout this thesis for parsers rather than matchers, wherever multiple constructions are expected to agree.

`MatcherType` (`src/matchers/selection.rs`) is an enum with one variant per matcher and a `matcher()` method returning the corresponding function pointer. Its purpose is purely to give the `regex-engine` command-line tool (`src/cli/matcher.rs`) a single place that maps a `--matcher` flag value (`naive`, `deriv`, `pderiv`, or `all`, defaulting to `deriv`) on the `match` subcommand to the matcher it names, rather than every call site duplicating that mapping itself. All three matchers can be compared side by side on an arbitrary input through the CLI directly, which is how the two worked examples of this chapter can be checked end to end rather than only by hand. Running each of x^* against `xx`, and r_{01} against `101` and `10`, with `--matcher all` gives:

Example	Word	naive	deriv	pderiv
x^*	<code>xx</code>	true	true	true
r_{01}	<code>101</code>	true	true	true
r_{01}	<code>10</code>	false	false	false

```
cargo run -- match "(0|1)*01" "101" --matcher all
```

prints `true` for all three matchers and reproduces the second row directly. This agrees with the hand-derived conclusions of Sections 3.2 and 3.3 in every cell: x^* accepts `xx`, r_{01} accepts `101`, and r_{01} rejects `10` and all three independently-written matchers reach each conclusion the same way. The same table-plus-CLI-invocation pattern is used again in Chapters 5 6 to check the parsers' worked examples against the `parse` subcommand instead of `match`.

Running these through the CLI does, however, lean on more than the three matchers themselves. `match_naive`, `match_deriv`, and `match_pderiv` take only an already-built `Regex` value and their shared signature is `fn(&str, &Regex) -> bool`, with no path from a string like `"(0|1)*01"` to a `Regex` anywhere in them. That translation is entirely the frontend's purpose: the `match` subcommand (`src/cli/matcher.rs`) calls `frontend::parse_pattern` *before* it calls whichever matcher `--matcher` selected, turning the concrete syntax into the `Regex` AST of Section 2.3 first. Chapter 7 covers that parsing and translation in detail.

The POSIX parsers built in Chapters 5 and 6 follow the same pattern one level up: rather than a Boolean, each parser returns a parse tree, but all of them remain selectable through a single uniform interface and are expected to agree with one another wherever they overlap.

Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$, the word is either accepted or rejected. Parsing refines this question by asking for the derivation that witnesses the match. Parse trees provide a representation of such derivations, recording the choices made by the structure of r while consuming w .

This refinement introduces ambiguity: a fixed r and w may admit multiple parse trees. A parser must therefore combine the semantics of matching with a *disambiguation policy* that selects one of them. This chapter introduces parse trees, examines Greedy and POSIX disambiguation, and establishes the ordering that the parsers developed later must compute.

4.1 The Matching Problem

The three matchers of Chapter 3 all answer the same Boolean question: given r and w , is $w \in L(r)$? Several applications need more than that answer. For example knowing *which alternative* of r matched, and knowing *which part* of w a given subexpression matched. Boolean recognition discards both kinds of information: it confirms that some derivation of the match exists, but nothing about what that derivation actually was [Frisch and Cardelli, 2004].

Which alternative: lexing and schema validation. Consider a lexer for a programming language, using the pattern `if|[a-z]+` to recognize a single token that could be the keyword `if`, marking the start of a conditional statement, or an ordinary lowercase identifier, naming a variable or function. Against the input `"if"`, both readings apply: the string is the literal keyword, but it is also a valid identifier, since it consists only of lowercase letters. A Boolean matcher confirms only that the pattern matches `"if"`; it does not say which of the two readings was taken, and the two readings are not interchangeable downstream. In a statement such as `if (x > 0) foo();`, the parser needs an `IF` token to recognize a conditional. If the lexer instead reported an identifier, `IDENT("if")`, the parser would try to read `if` as an expression, and the conditional would never be recognized.

In practice, this particular ambiguity is resolved before parsing ever begins, not during it. Most languages forbid identifiers from being spelled the same as a keyword, and lexer generators enforce this directly: flex always matches the longest possible token, and when two rules match input of the same length, the rule listed earlier in the specification wins [The Flex Project]. Placing the keyword pattern for `if` before the general identifier pattern is therefore enough to guarantee the keyword reading whenever both would otherwise apply,

and "if" can never be tokenized as an identifier at all. The underlying problem, that a Boolean matcher cannot report which alternative explains a match, has not gone away; it has been settled in advance by a rule the tool applies uniformly, rather than recovered from any particular match. That rule is already a disambiguation policy in miniature, of exactly the kind Section 4.3.1 and Section 4.3.3 formalize later in this chapter.

Schema validation avoids the same problem whenever a schema declares a field's type up front: checking a value declared as an integer against the pattern for integers never raises the question of whether it might also be a Boolean, since no other pattern is ever consulted. The problem reappears in formats that infer a value's type from its text instead of from a declared schema. YAML is the clearest documented case. A plain, unquoted scalar is resolved to a type by checking it against an ordered set of patterns, one for null, one for booleans, one for integers, one for floats, and so on, with the type belonging to the first pattern that matches [Ben-Kiki et al.]. In YAML 1.1, the boolean pattern accepts not only `true` and `false` but also `yes`, `no`, `on`, `off`, and several capitalizations of each, so the two-letter ISO country code for Norway, `NO`, is resolved to the Boolean value `false` rather than the string `"NO"` in any YAML document that does not quote it explicitly. This became widely known as the "Norway problem," and it is exactly the failure this section describes: the text `NO` matches both the boolean pattern and, trivially, the fallback string pattern, and only the fixed order in which YAML checks its candidate types, not anything about the text `NO` itself, decides which one wins [HitchDev]. YAML 1.2 removed the `yes/no/on/off` boolean forms specifically to shrink this overlap, though widely used implementations such as PyYAML still default to the 1.1 rules.

A pattern where the two readings genuinely compete. The "if" example is resolved by a tie-breaking convention. Both alternatives of `if|[a-z]+` match the same two-character input, so a longest-match rule cannot distinguish them. The lexical specification therefore gives the keyword alternative priority. What appears to be an obvious choice is thus the result of a specific disambiguation rule: the two alternatives consume equally long substrings, and the earlier alternative wins the tie.

The pattern $(a+ab)(b+\varepsilon)$ against "ab" presents a different situation. The two alternatives of $a + ab$ consume different amounts of input. The `ab` alternative can consume both characters, after which $(b + \varepsilon)$ matches ε . Alternatively, the `a` alternative consumes only the first character, leaving `b` to be matched by the `b` alternative of $(b + \varepsilon)$. Both choices produce a complete and valid match of the same input.

Here, two natural disambiguation policies give different results. A policy that prefers the longer match selects the `ab` alternative, whereas a policy that gives priority to the earlier alternative selects `a`. Unlike the "if" example, neither choice can be dismissed as merely resolving a tie. The expression admits both parses, and selecting between them requires an explicit policy.

Before such choices can be compared, however, the possible parses must be made explicit. A Boolean result records only that a match exists; it does not record the structural choices through which the match was obtained. Section 4.2 represents these choices as *parse trees* for a structured *witness* to a match: it records how the structure of r was used to consume w . When multiple parse trees witness the same match, the parser must determine which one to return.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu formalize such a witness by viewing regular expressions as types and parse trees as the values inhabiting them [Sulzmann and Lu, 2014]. A parse tree v is built from the same constructors used informally in Section 2.2’s definition of $L(r)$: a pair for concatenation, a tagged left or right injection for alternation, and a list for repetition. A judgment $\vdash v : r$, defined by structural recursion on both v and r , states that v is a valid parse tree for r :

$$\begin{array}{c}
\vdash () : \varepsilon \qquad \frac{l \in \Sigma}{\vdash l : l} \\
\\
\frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \\
\\
\frac{}{\vdash [] : r^*} \quad \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*}
\end{array}$$

A *flattening* function $|v|$ reads the word represented by a parse tree by structural recursion:

$$\begin{aligned}
|()| &= \varepsilon, & |l| &= l, & |[]| &= \varepsilon, \\
|(v_1, v_2)| &= |v_1||v_2|, & |\text{Left } v| &= |v|, & |\text{Right } v| &= |v|, \\
|v : vs| &= |v||vs|.
\end{aligned}$$

The key property is that the parse trees characterize the language:

$$L(r) = \{ |v| \mid \vdash v : r \}.$$

Thus, a parse tree together with its derivation provides a structured *witness* that $w \in L(r)$: it records not only that the match exists, but the choices made by the regular expression while producing w .

Unambiguous example. For an unambiguous match, this witness is unique. For $r = a \cdot b$ and $w = \mathbf{ab}$, only one derivation is possible. The *Pair* rule requires $\vdash v_1 : a$ and $\vdash v_2 : b$, and the only parse tree for each literal is the literal itself. Hence $v_1 = a$ and $v_2 = b$, giving

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}.$$

This is the only possible derivation, yielding $v = (a, b)$ with

$$|v| = |(a, b)| = |a||b| = \mathbf{ab}.$$

When the structure of the expression permits different choices, the witness itself can be ambiguous. The first two examples below are adapted from the running examples of Sulzmann and Lu [2014] and are reproduced in this project’s test suite as `paper_r1` and `paper_r2`. They are included here because the same cases are used later to illustrate

and evaluate the parser implementations developed in Chapters 5 and 6. The derivations given here provide a reference for that later discussion. Two further examples, drawn from this project's own test suite, follow.

Ambiguous example 1: concatenation. Consider $r_1 = (a + ab)(b + \varepsilon)$ against $w = \mathbf{ab}$. One valid derivation chooses the second alternative of $a + ab$, which consumes both characters, and the empty alternative of $b + \varepsilon$:

$$\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon}$$

and therefore

$$\vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon).$$

The corresponding parse tree is

$$v = (\text{Right}(a, b), \text{Right}()),$$

with

$$|v| = |(a, b)| |()| = \mathbf{ab} \varepsilon = \mathbf{ab}.$$

This is not the only witness. The first alternative of $a + ab$ can instead consume $\mathbf{a}$, leaving $\mathbf{b}$ for the first alternative of $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon}$$

and therefore

$$\vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon).$$

The corresponding parse tree is

$$v' = (\text{Left } a, \text{Left } b),$$

with

$$|v'| = |a| |b| = \mathbf{ab}.$$

Thus the same expression r_1 and word w have two distinct parse trees, v and v' , satisfying

$$|v| = |v'| = \mathbf{ab}.$$

Ambiguous example 2: repetition. The same phenomenon can arise from repetition rather than concatenation. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, nesting the alternation as $a + (b + ab)$ (`paper_r2`¹ in the test suite). One derivation uses a single iteration, taking the `ab` alternative directly:

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right Right}(a, b) : a + (b + ab)}$$

which becomes a one-element list via the star’s constructor rule:

$$\vdash [\text{Right Right}(a, b)] : (a + (b + ab))^*.$$

The corresponding parse tree is

$$v_2 = [\text{Right Right}(a, b)], \quad |v_2| = |(a, b)| = \mathbf{ab}.$$

A second derivation instead uses two iterations, matching `a` and `b` through separate alternatives of the star’s body:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right Left } b : a + (b + ab)}$$

which combine, one iteration at a time, into

$$\vdash [\text{Left } a, \text{Right Left } b] : (a + (b + ab))^*.$$

The corresponding parse tree, with two iterations of the star, is

$$v'_2 = [\text{Left } a, \text{Right Left } b], \quad |v'_2| = |a| |b| = \mathbf{ab}.$$

As with r_1 , both v_2 and v'_2 witness the same match: ambiguity here comes from how many iterations the star takes, not from which top-level alternative fires.

Ambiguous example 3: duplicate alternatives. Concatenation and repetition are not the only sources of ambiguity. Consider $r_3 = (a + a)^*$ against $w = \mathbf{aa}$. Here the star’s body contains two syntactically distinct but semantically identical alternatives for the same literal. This is an instance of `ambiguous_star_dedup_matches_bitcoded` in `src/parsers/pderiv_std/parse.rs`².

Each of the two positions in w can independently be matched by either the left or the right copy of `a`, giving four distinct parse trees for this two-character input alone:

¹The associativity of $+$ is immaterial to $L(r)$ and to this example.

²The test uses a four-character input; the two-character case shown here exhibits the same phenomenon.

$[Left\ a, Left\ a],\ [Left\ a, Right\ a],\ [Right\ a, Left\ a],\ [Right\ a, Right\ a],$

all flattening to aa . Nothing about *Left a* and *Right a* distinguishes them semantically; the ambiguity is entirely an artifact of writing the same alternative twice. This kind of duplication is particularly relevant to the partial-derivative parser, where distinct syntactic paths can produce equivalent residual matches. The deduplication mechanisms discussed in Chapter 6 prevent such redundant paths from accumulating in the parse frontier.

Ambiguous example 4: an IPv4 pattern. The previous examples isolate particular sources of ambiguity. The same phenomenon also occurs in a pattern taken from practice: a regular expression for dotted-quad IPv4 addresses from the Snort intrusion-detection dataset studied by Mamouras et al. [2024]. The pattern is included in this project’s test suite as `snort_ip_regex`³.

$$((\backslash d[1-9]\backslash d|1\backslash d\backslash d|2[0-4]\backslash d|25[0-5])\backslash .)^{\{3\}} \\ (\backslash d[1-9]\backslash d|1\backslash d\backslash d|2[0-4]\backslash d|25[0-5])$$

using the class and repetition shorthands of Section 7.3, under which $\backslash d$ desugars to a ten-way alternation of the literals 0 through 9 and $\{3\}$ desugars to three concatenated copies of its operand. The pattern concatenates four copies of a shared three-character octet pattern, separated by literal dots. Write $O = A + B + C + D$ for this shared four-way alternation, with

$$A = \backslash d[1-9]\backslash d \quad B = 1\backslash d\backslash d \quad C = 2[0-4]\backslash d \quad D = 25[0-5],$$

left-associated as $((A + B) + C) + D$, so that the full pattern is $(O.)^3O$ with the final dot omitted. Already at the level of a single octet, O is ambiguous on some inputs in exactly the sense of r_1 : against $w = 239$, alternative A matches, reading the digits as *any digit, a digit from 1 to 9, any digit*; but so does alternative C , reading 2 as the literal 2, 3 as a digit from 0 to 4, and 9 as any digit (B and D both fail, since w does not begin with 1 or 25). Both A and C consume all three characters of w . Thus, unlike r_3 , where the alternatives are semantically identical, the ambiguity here is between two genuinely different alternatives that happen to match the same substring.

4.3 Disambiguation Policies

The parse-tree judgment identifies every valid witness, but it does not say which witness a parser should return. A *disambiguation policy* adds this missing choice. Given a regular expression r and word w , the policy selects one parse tree v satisfying

$$\vdash v : r \quad \text{and} \quad |v| = w.$$

The four ambiguous examples above illustrate why the policy matters, and in more than one way. For r_1 and ab , one witness chooses a before b , while the other assigns the whole input to ab . For r_2 and ab , one witness uses two iterations and the other uses one. For r_3 and aa , four witnesses differ only in which of two identical alternatives each position is attributed to. For O and 239 , two alternatives with different content, not merely different

³`tests/test_frontend.rs`.

labels, both happen to consume the whole octet. Different matching conventions select different witnesses, and, as r_3 and O show, a policy also has to say something about witnesses that agree on every character consumed and differ only in which alternative gets the credit.

The two policies considered here resolve these choices differently. Greedy matching follows the structural priority of the expression, whereas POSIX matching gives priority to longer leftmost submatches. Their difference becomes clear when they are applied to the same ambiguous expression.

4.3.1 Greedy Leftmost Matching

The policy associated with conventional backtracking matchers is *Greedy*. Alternatives are considered from left to right, and the first alternative that leads to an overall match is selected. Within a Kleene star, another iteration is preferred to stopping whenever continuing the repetition can still lead to an overall match [Frisch and Cardelli, 2004].

For alternatives, the defining preference is structural: the left alternative is preferred regardless of how much input the two alternatives consume. In an ordering notation, the essential rule is

$$Left\ v \succ_{r_1+r_2} Right\ v'.$$

Applied to r_1 . Greedy first tries the **a** alternative. It consumes the leading **a**, and the remaining **b** is successfully matched by the first alternative of $b + \varepsilon$. Since the complete match succeeds, there is no need to try the **ab** alternative. The Greedy result is therefore

$$v' = (Left\ a, Left\ b).$$

Applied to r_2 . Greedy similarly begins with the first alternative, matching **a**. The star then attempts another iteration, in which the first alternative fails on **b** and the second alternative succeeds. The resulting parse uses two iterations:

$$v'_2 = [Left\ a, Right\ Left\ b].$$

The Greedy result therefore depends directly on the order in which the regular expression presents its alternatives.

Applied to r_3 . At each of the two positions, $Left\ v \succ_{a+a} Right\ v'$ applies directly, regardless of length, since both alternatives consume the same single character. Greedy therefore selects the left alternative every time:

$$[Left\ a, Left\ a].$$

Unlike r_1 and r_2 , this result is not decided by which alternative matches more of the input; every alternative here matches exactly one character. It is decided purely by position in the pattern, the same kind of tie-breaking that Section 4.1 showed resolving "if" against $if\ |\ [a-z]^+$.

Applied to O . Against $w = 239$, Greedy tries O 's alternatives left to right: A , then B , then C , then D . Alternative A matches the leading digit 2 against `\d`, the digit 3 against `[1-9]`, and the digit 9 against `\d`, consuming all three characters. Since this already produces a complete match, Greedy never tries B , C , or D , even though C would also have matched. The Greedy result selects A , exactly as it selected the left alternative of r_1 and of r_3 : once an alternative leads to a full match, later alternatives are never even attempted.

4.3.2 The Infinite Empty-Match Chain Problem

Greedy's preference for continuing a repetition creates a problem when the repeated expression can match the empty word. Consider ε^* against the empty input. The following parse trees are all valid:

$$v_0 = [], \quad v_1 = [()], \quad v_2 = [(), ()], \quad \dots$$

Each additional iteration still produces the same flattened word ε . Under the Greedy ordering, every additional iteration is preferred:

$$v_0 \prec_{\varepsilon^*} v_1 \prec_{\varepsilon^*} v_2 \prec_{\varepsilon^*} \dots$$

There is therefore no maximal parse tree. The problem is not that the language is ill-defined; ε is clearly a valid match. The problem is that the Greedy preference does not identify a largest witness when empty iterations can be added indefinitely.

A slightly less degenerate example is $(\varepsilon + a)^*$ against **a**. The parse trees

$$\begin{aligned} v_0 &= [Right\ a], \\ v_1 &= [Left\ (), Right\ a], \\ v_2 &= [Left\ (), Left\ (), Right\ a], \\ &\vdots \end{aligned}$$

all flatten to **a**. Greedy prefers each additional empty iteration, producing another infinite ascending chain.

The same pathology reappears whenever a star's body is itself nullable, not just when the body is ε outright. Take $(a^*)^*$ against **aaa**: since the inner a^* can always match ε , an outer iteration contributing nothing is always available, and Greedy always prefers taking it:

$$[[a, a, a]] \prec_{(a^*)^*} [[a, a, a], []] \prec_{(a^*)^*} [[a, a, a], [], []] \prec_{(a^*)^*} \dots,$$

another infinite ascending chain, this time built on top of an otherwise unambiguous inner match rather than on ε itself. Nested, mutually nullable stars of this shape are also the classic trigger for catastrophic backtracking in engines that search for a Greedy-maximal match directly [Cox, 2007]; the ordering-theoretic failure identified here is the same failure that shows up operationally as exponential blowup in those engines. The

frontier-based Greedy parsers of Chapter 6 do not search for an ordering maximum at all, so this particular failure mode does not resurface there; Section 4.3.3 returns to $(a^*)^*$ once POSIX’s rule (K1) is available, to show why POSIX does not have this problem either.

This exposes a limitation of the Greedy ordering: without additional restrictions or special handling of empty matches, a maximal parse tree need not exist. The POSIX ordering avoids this problem by treating empty matches explicitly.

4.3.3 POSIX Longest-Leftmost Matching

POSIX uses a different disambiguation policy. Subpatterns are compared according to the length of the substrings they match, with subpatterns that occur earlier in the regular expression taking priority. The standard describes this as giving subpatterns the “longest possible substrings” while giving earlier subpatterns priority over later ones [IEEE and The Open Group, 2017].

Sulzmann and Lu formalize this policy as an ordering on parse trees [Sulzmann and Lu, 2014]. Let $\text{len}(w)$ denote the number of characters in w . The ordering is defined by the following rules:

$$\begin{aligned}
\text{(C1)} \quad & \frac{v_1 = v'_1 \quad v_2 \succ_{r_2} v'_2}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(C2)} \quad & \frac{v_1 \succ_{r_1} v'_1}{(v_1, v_2) \succ_{r_1 r_2} (v'_1, v'_2)} \\
\text{(A1)} \quad & \frac{\text{len}(|v_2|) > \text{len}(|v_1|)}{\text{Right } v_2 \succ_{r_1+r_2} \text{Left } v_1} \\
\text{(A2)} \quad & \frac{\text{len}(|v_1|) \geq \text{len}(|v_2|)}{\text{Left } v_1 \succ_{r_1+r_2} \text{Right } v_2} \\
\text{(A3)} \quad & \frac{v_2 \succ_{r_2} v'_2}{\text{Right } v_2 \succ_{r_1+r_2} \text{Right } v'_2} \\
\text{(A4)} \quad & \frac{v_1 \succ_{r_1} v'_1}{\text{Left } v_1 \succ_{r_1+r_2} \text{Left } v'_1} \\
\text{(K1)} \quad & \frac{|v : vs| = \varepsilon}{[] \succ_{r^*} v : vs} \\
\text{(K2)} \quad & \frac{|v : vs| \neq \varepsilon}{v : vs \succ_{r^*} []} \\
\text{(K3)} \quad & \frac{v_1 \succ_r v_2}{v_1 : vs_1 \succ_{r^*} v_2 : vs_2} \\
\text{(K4)} \quad & \frac{v_1 = v_2 \quad vs_1 \succ_{r^*} vs_2}{v_1 : vs_1 \succ_{r^*} v_2 : vs_2}
\end{aligned}$$

The corresponding non-strict relation $\succeq_r$ is defined by

$$v_1 \succeq_r v_2 \iff v_1 = v_2 \quad \text{or} \quad v_1 \succ_r v_2.$$

Rules (C1) and (C2) propagate comparisons through concatenation. Rule (C1) compares the second component when the first components are equal, while (C2) gives priority to the first component whenever it differs. Thus, earlier subexpressions have priority over later ones.

Rules (A1)–(A4) define the crucial difference from Greedy matching. When two alternatives compete, the alternative producing the longer substring is preferred. If the two substrings have the same length, (A2) gives preference to the left alternative. Rules (A3) and (A4) continue the comparison when both candidates have already chosen the same alternative.

Rules (K1)–(K4) apply the same idea to repetition. In particular, (K1) prefers the empty list when a non-empty sequence of iterations also flattens to ε , while (K2) prefers a non-empty match over the empty list. Rules (K3) and (K4) compare the iterations structurally, first comparing their heads and then their tails.

Applied to r_1 . Return to $r_1 = (a + ab)(b + \varepsilon)$ against $\mathbf{ab}$, with

$$v = (Right(a, b), Right())$$

and

$$v' = (Left\ a, Left\ b).$$

The two parse trees already choose different alternatives within $a + ab$, so rule (C2) applies directly, reducing the comparison to exactly those first components,

$$Right(a, b) \quad \text{versus} \quad Left\ a$$

within $a + ab$. Rule (A1) applies because

$$\text{len}(|(a, b)|) = 2 > 1 = \text{len}(|a|).$$

Hence

$$Right(a, b) \succ_{a+ab} Left\ a,$$

and therefore

$$v \succeq_{r_1} v'.$$

POSIX consequently selects

$$v = (Right(a, b), Right()),$$

which is the opposite of the Greedy result.

Applied to r_2 . For $r_2 = (a + b + ab)^*$ against **ab**, consider

$$v_2 = [Right\ Right(a, b)]$$

and

$$v'_2 = [Left\ a, Right\ Left\ b].$$

Rule (K3) compares the first iterations within the body $a + (b + ab)$:

$$Right\ Right(a, b) \quad \text{versus} \quad Left\ a.$$

Again, (A1) applies because the first candidate consumes two characters while the second consumes one:

$$\text{len}(|Right(a, b)|) = 2 > 1 = \text{len}(|a|).$$

Thus

$$Right\ Right(a, b) \succ_{a+(b+ab)} Left\ a,$$

and (K3) lifts this comparison to the star:

$$v_2 \succeq_{r_2} v'_2.$$

POSIX therefore selects the single two-character iteration, whereas Greedy selects the two one-character iterations.

Applied to r_3 . For $r_3 = (a + a)^*$ against **aa**, every candidate alternative at every position matches exactly one character, so (A1)'s strict inequality never holds and every comparison falls to (A2)'s tie-break:

$$\text{len}(|a|) \geq \text{len}(|a|) \implies Left\ a \succ_{a+a} Right\ a.$$

(K3) and (K4) lift this position by position, giving the same result as Greedy:

$$[Left\ a, Left\ a].$$

Unlike r_1 and r_2 , Greedy and POSIX agree here. This is not a coincidence: (A2) only ever overrules (A1) when both alternatives tie in length, and a tie is exactly the condition under which Greedy's unconditional left-preference and POSIX's tie-broken left-preference compute the same thing, the same connection already observed for "if" in Section 4.1.

Applied to O . For $O = ((A + B) + C) + D$ against $w = 239$, only A and C match at all, so the comparison is between a tree choosing A and a tree choosing C . Both trees agree on the outer alternation: neither chooses D , so the comparison reduces, by (A4), to a comparison within $(A + B) + C$ of a tree choosing A (further inside $A + B$) against one choosing C directly. Since A and C both consume the full input,

$$\text{len}(|v_A|) = 3 = \text{len}(|v_C|),$$

so (A1)'s strict inequality fails and (A2) resolves the tie in favor of the left-hand candidate, the one going through $A + B$ and ultimately choosing A . POSIX therefore also selects A . As with r_3 , this is not a coincidence: A and C tie in length, so (A2)'s tie-break coincides with Greedy's unconditional left-preference, even though A and C , unlike r_3 's two copies of $\mathbf{a}$, are not otherwise interchangeable.

Applied to the infinite-chain examples. Rule (K1) resolves Section 4.3.2's pathology directly: for ε^* against ε , since

$$|() : []| = |()| |[]| = \varepsilon,$$

(K1) gives

$$[] \succ_{\varepsilon^*} [()].$$

The same rule applies to every longer list of empty iterations. Thus $[]$ is preferred to every parse tree obtained by repeatedly matching ε .

For $(\varepsilon + a)^*$ against $\mathbf{a}$, the relevant comparison is different. Consider

$$v_0 = [Right\ a]$$

and

$$v_1 = [Left\ (), Right\ a].$$

Rule (K3) compares their first iterations, reducing the comparison to

$$Right\ a \quad \text{versus} \quad Left\ ()$$

within $\varepsilon + a$. Rule (A1) applies because

$$\text{len}(|a|) = 1 > 0 = \text{len}(|()|).$$

Therefore

$$Right\ a \succ_{\varepsilon+a} Left\ (),$$

and hence

$$v_0 \succ_{(\varepsilon+a)^*} v_1.$$

The same comparison applies to every parse tree obtained by inserting additional empty iterations before the iteration matching **a**. Thus the POSIX ordering selects v_0 , while the Greedy ordering admits the infinite ascending chain.

Finally, for $(a^*)^*$ against **aaa**, compare

$$[[a, a, a]] \quad \text{against} \quad [[a, a, a], []].$$

Since $[] : [] = \varepsilon$, (K1) applies to the tail once the first (identical) iteration is matched by (K4), giving

$$[[a, a, a]] \succ_{(a^*)^*} [[a, a, a], []],$$

and the same argument disposes of every list padded with further empty iterations after the first. POSIX therefore selects the single outer iteration whose inner star consumes all three characters directly,

$$[[a, a, a]],$$

exactly the same tree the running implementation returns for $(\mathbf{a}^*)^*$ against **aaa** under `deriv_std_rec` and `deriv_bc`. Unlike the abstract Greedy ordering, which has no maximal element for this expression, the frontier-based Greedy implementations of Chapter 6 do not compute an ordering maximum at all, and return this same tree without difficulty; the ordering-theoretic failure identified in Section 4.3.2 is a property of the abstract Greedy comparison, not of every construction that happens to compute a Greedy answer.

The contrast between the two policies is now explicit. Greedy resolves ambiguity according to the structural priority encoded by the regular expression, whereas POSIX compares the substrings assigned to subexpressions and gives priority to longer leftmost matches. The POSIX ordering of Sulzmann and Lu is designed to select a maximal parse tree for every expression and input [Sulzmann and Lu, 2014]. The derivative parser developed in Chapter 5 computes that candidate without enumerating all possible parse trees.

Derivative-Based Parser

Chapter 4 fixed what is being computed – the $\geq_r$ -maximal parse tree of Section 4.3.3 – without saying how. This chapter presents Sulzmann and Lu’s answer [Sulzmann and Lu, 2014]: a parser built directly on the Brzozowski derivative of Section 3.2, in two forms. The first, direct construction computes a derivative sequence and then explicitly rebuilds a parse tree from it in a second, backward pass; the second, bit-coded construction fuses parse-tree bookkeeping into the derivative step itself, needing only one forward pass. Both are POSIX – they compute the identical $\geq_r$ -maximal tree, verified by direct comparison in Chapter 8 – and differ only in how that tree is produced. Both are developed here as theory first (Sections 5.1-5.4), then as the two Rust implementations that realize them, `deriv_std` (Section 5.5) and `deriv_bc` (Section 5.6). Chapter 6 repeats the exercise once more, replacing the Brzozowski derivative throughout with Antimirov’s partial derivative from Section 3.3 – and, as that chapter shows, changing more than just which derivative operator is used.

5.1 Core Algorithm

The direct construction follows the same two-phase shape informally sketched for Brzozowski’s derivative in Section 3.2: consume the input by taking successive derivatives, then decide whether the result is nullable. What is new here is what happens *between* those two phases – instead of a single Boolean, a parse tree is built. The construction is best understood in two halves: a forward pass that is nothing more than the derivative sequence already introduced, and a backward pass that walks that sequence in reverse to assemble a tree one consumed character at a time.

Forward pass. For input $x_1x_2\cdots x_n$, compute $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ exactly as before, where $r_0 = r$ and $r_i = r_{i-1} \setminus x_i$. If r_n is not nullable, no parse tree exists and parsing fails. This half of the construction is identical to `match_deriv` (Section 3.2) with one difference: no simplification is applied between steps. `simplify` is safe for a Boolean matcher, since it only ever normalizes r_i to a language-equivalent expression, but the backward pass below needs to walk back through the *exact* sequence of derivative transformations that produced r_n from r_0 – a simplified r_i may no longer be the expression `inject` expects to see at that step.

Backward pass. Once r_n is known to be nullable, the parse tree v_n for the empty remainder is built directly, by a function `mkEps` defined by structural recursion on a

nullable expression:

$$\begin{aligned}
mkEps_\varepsilon &= () & mkEps_{r^*} &= [] \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & mkEps_{r_1+r_2} &= \begin{cases} \textit{Left } mkEps_{r_1} & \varepsilon \in L(r_1) \\ \textit{Right } mkEps_{r_2} & \text{otherwise} \end{cases}
\end{aligned}$$

$mkEps$ has no case for a literal x or for $\emptyset$, since neither is ever nullable – the function is only ever called on an expression already known to satisfy $n(r)$. Its only interesting case is alternation: exactly one of rule (A1)/(A2) of Section 4.3.3 is relevant here, since both sides of an empty match have length 0 – (A2)’s tie-break, "prefer left when lengths are equal", degenerates at the empty word to "prefer left whenever left is even possible", which is exactly the $\varepsilon \in L(r_1)$ guard above.

With v_n in hand, the remaining task is to turn it back into a parse tree of the *original* r_0 , one derivative step in reverse at a time. This is the job of inj , an injection function that takes a parse tree of a derivative $r \setminus x$ and re-inserts the consumed character x to produce a parse tree of r itself:

$$\begin{aligned}
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x} v) : vs \\
inj_{(r_1 r_2) \setminus x}(v) &= \begin{cases} (inj_{r_1 \setminus x} v_1, v_2) & v = (v_1, v_2), \text{ not nullable}(r_1) \\ (inj_{r_1 \setminus x} v_1, v_2) & v = \textit{Left}(v_1, v_2), \text{ nullable}(r_1) \\ (mkEps_{r_1}, inj_{r_2 \setminus x} v_2) & v = \textit{Right } v_2, \text{ nullable}(r_1) \end{cases} \\
inj_{(r_1+r_2) \setminus x}(v) &= \begin{cases} \textit{Left } (inj_{r_1 \setminus x} v_1) & v = \textit{Left } v_1 \\ \textit{Right } (inj_{r_2 \setminus x} v_2) & v = \textit{Right } v_2 \end{cases} \\
inj_{x \setminus x}() &= x
\end{aligned}$$

The three-way $(r_1 r_2)$ case is worth reading against the derivative rule it inverts (Section 3.2): $(r_1 r_2) \setminus x$ produced either $(r_1 \setminus x) r_2$ alone (when r_1 is not nullable) or that same term joined with $r_2 \setminus x$ via $+$ (when it is), tagged *Left/Right* respectively by the *Alt* this new $+$ introduces. inj undoes exactly that tagging: a bare pair means r_1 was not nullable and the derivative went straight into it; a *Left*-tagged pair means it was nullable but the derivative still chose to continue matching r_1 ; a *Right*-tagged value means the derivative instead jumped straight into r_2 , so r_1 ’s own contribution to the reconstructed pair has to be synthesized fresh via $mkEps_{r_1}$, since no part of it was consumed by x at all.

Parsing is then this two-phase process packaged into one function, applying $mkEps$ once at the end of the forward pass and inj once per character on the way back:

$$\textit{parse}(r, \varepsilon) = mkEps_r \quad (\text{if } \varepsilon \in L(r)) \quad \textit{parse}(r, xw) = inj_{r \setminus x}(\textit{parse}(r \setminus x, w))$$

Worked example. Return to $r_1 = (a + ab)(b + \varepsilon)$ against **ab**, whose two candidate parse trees were already established in Section 4.3. The forward pass computes $r_1 \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi)$, nullable via its right disjunct $(\varepsilon + \phi)$. The backward pass starts from $mkEps$ of that final expression – $\textit{Right}(\textit{Right}(), \textit{Right}())$, abbreviated $(\textit{Right}(), \textit{Right}())$ once the outermost tag (there is only one alternative left at the top level once ϕ -branches are excluded) is elided – then injects **b** and finally **a** back in, arriving at $(\textit{Right}(a, b), \textit{Right}())$: the *longer* ab branch, matching the POSIX answer already claimed in Section 4.3.3 rather than Greedy’s $(\textit{Left } a, \textit{Left } b)$.

5.2 Correctness Properties

Sulzmann and Lu prove three results establishing that *parse* computes exactly the $\geq_r$ -maximal tree of Definition 4.3.3, not merely a valid one [Sulzmann and Lu, 2014]. This section states them; full proofs are in the reference and are not reproduced here, but the intuition behind each is worth making explicit, since it is exactly what justifies trusting the Rust implementation of Section 5.5 beyond "it agrees with the reference on every test case we tried" (Chapter 8 verifies that too, but the proofs are why agreement on finitely many test cases is expected to generalize).

Lemma 1 (Empty POSIX Parse Tree). For r with $\varepsilon \in L(r)$, $mkEps_r$ is the POSIX parse tree of r for the empty word. This grounds the base case of the induction the other two results build on: at $mkEps$'s only interesting case, alternation, both sides can only ever match length 0 (there is no remaining input to consume), so (A2)'s length comparison always ties and reduces to left-preference – exactly the $\varepsilon \in L(r_1)$ test $mkEps$ performs.

Lemma 2 (POSIX Preservation under Injection). If v is the POSIX parse tree of $r \setminus x$, then $inj_{r \setminus x}(v)$ is the POSIX parse tree of r . This is the result doing the real work, and its proof is correspondingly the most involved of the three. The informal argument: the derivative operation always tries r_1 before falling through to r_2 at a concatenation or alternation node (Section 3.2's $(r_1 r_2) \setminus x$ case only reaches $s \setminus x$ when r_1 is nullable, and even then keeps the r_1 -continuation branch as the left disjunct of the resulting $+$); inj reconstructing that exact same left-first bias at every backward step, rather than an arbitrary one, is what carries POSIX-maximality from the smaller expression $r \setminus x$ up to the original r .

Theorem 1 (POSIX Parsing). *parse* computes POSIX parse trees. This follows from Lemmas 1 and 2 by induction on the input word: the base case is Lemma 1 applied to r_n , and each inductive step is one application of Lemma 2, carrying POSIX-maximality from $r \setminus x_1 \cdots x_n$ back up through n injections to r itself.

5.3 Bit-Coded Optimization (Theory)

The direct construction of Section 5.1 must retain the entire forward-pass sequence $r_0, r_1, \dots, r_n$ (or, in an implementation, an equivalent call stack) so that inj has each $r_{i-1} \setminus x_i$ available to reconstruct against on the way back – the backward pass cannot begin until the forward pass has fully finished. Sulzmann and Lu's bit-coded construction (their Section 3.2, "Incremental Bit-Coded Forward Parse Tree Construction") [Sulzmann and Lu, 2014] removes this requirement by incrementally building the parse tree *during* the forward pass itself, so that by the time the last character is consumed, the winning tree's information is already sitting in the final expression – no second pass required.

Bit-code representation. Rather than a `ParseTree` value, a parse tree is represented as a flat sequence of bits: 0 marks a *Left* choice, 1 a *Right* choice, and inside a Kleene star, 0 marks "one more iteration" and 1 marks "stop". A pair of mutually recursive functions $encode_r/decode_r$ convert between a $\vdash v : r$ parse tree and its bit-string representation, structurally straightforward given the tagging already present in v itself.

Annotated regular expressions. The construction's core idea is to annotate the derivative operation's own *input*, not just its output, with partial bit-code information, so that by the time a sub-expression becomes ε its accumulated annotation already *is* the relevant slice of the final bit-string. An annotated regular expression ri extends plain r with a bit-list bs attached at every node except $\emptyset$ (which denotes no language and so has no parse tree to annotate):

$$ri ::= \emptyset \mid (bs@ \varepsilon) \mid (bs@ x) \mid (bs@ ri_1 \oplus ri_2) \mid (bs@ ri_1 ri_2) \mid (bs@ ri^*)$$

Two operations connect plain r to this annotated world. *internalize* converts r into ri with every bit-list initially empty, except at *Alt*: the left operand of $r_1 + r_2$ is tagged with a leading 0 and the right with a leading 1 at construction time, so that which alternative a sub-tree came from can always be recovered from its own annotation without re-inspecting the surrounding structure. *fuse*(bs, ri) prepends a bit-list onto whatever annotation ri 's own top-most node already carries – the one primitive every other operation in this construction is built from.

The refined derivative. The ordinary derivative operation is lifted to annotated expressions, written $ri \backslash_b l$, following the same case structure as $r \backslash l$ (Section 3.2) but threading bits through instead of discarding the annotation:

$$\begin{aligned} (bs@ ri_1 \oplus ri_2) \backslash_b l &= (bs@ (ri_1 \backslash_b l) \oplus (ri_2 \backslash_b l)) \\ (bs@ ri_1 ri_2) \backslash_b l &= \begin{cases} (bs@ (ri_1 \backslash_b l) ri_2) \oplus \text{fuse}(mkEpsBC_{ri_1}, ri_2 \backslash_b l) & \varepsilon \in L(ri_1) \\ (bs@ (ri_1 \backslash_b l) ri_2) & \text{otherwise} \end{cases} \\ (bs@ ri^*) \backslash_b l &= (bs@ \text{fuse}([0], ri \backslash_b l) ([]@ ri^*)) \end{aligned}$$

The concatenation case mirrors $(r_1 r_2) \backslash x$'s two-way split exactly, with one addition: on the branch where r_1 's derivative is abandoned in favor of jumping into r_2 , the bits r_1 would have contributed to an empty match ($mkEpsBC_{ri_1}$, below) are fused directly onto r_2 's derivative rather than discarded – this is the incremental step that replaces the direct construction's *later* call to $mkEps_{r_1}$ inside *inj* (Section 5.1) with work done immediately, during the forward pass. The star case attaches a leading 0 bit ("one more iteration") each time the body is derived once more; the outer bs stays where it is, on the new alternation/concatenation node the derivative rule produces, exactly as it does for plain *Alt*.

mkEpsBC. Mirroring *mkEps* (Section 5.1) but returning accumulated bits instead of a tree:

$$\begin{aligned} mkEpsBC_{(bs@ \varepsilon)} &= bs \\ mkEpsBC_{(bs@ ri_1 \oplus ri_2)} &= bs ++ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise} \end{cases} \\ mkEpsBC_{(bs@ ri_1 ri_2)} &= bs ++ mkEpsBC_{ri_1} ++ mkEpsBC_{ri_2} \\ mkEpsBC_{(bs@ ri^*)} &= bs ++ [1] \end{aligned}$$

Once the forward pass ends on a nullable annotated expression, *mkEpsBC* extracts its complete bit-string directly, and *decode_r* (using the *original*, un-annotated r – *decode*

needs to know the shape it is decoding bits *for*, which the annotated ri alone does not by itself determine once its own annotations have been consumed) turns that bit-string into an ordinary parse tree. No second, backward pass over the input is needed at all.

Worked example. Run Section 5.1's $r_1 = (a + ab)(b + \varepsilon)$ against `ab` through this construction instead, to see the same POSIX answer assembled incrementally rather than reconstructed afterward (this trace is exactly what `cargo run - parse "(a|ab)(b|ε)" "ab" -parser deriv_bc -diag 3` prints; the tool's own `false/true` bits are written 0/1 below to match this section's convention). `internalize(r1)` tags the two *Alt* nodes' operands with 0/1 and leaves every other bit-list empty:

$$ri_0 = []@([]@(0@a \oplus 1@([]@a \cdot []@b)) \cdot []@(0@b \oplus 1@ε))$$

Deriving with respect to **a**: the left *Seq* operand $[]@(0@a \oplus 1@ab)$ is not nullable, so the *Seq* case's first branch applies, threading the derivative into it alone and leaving $(b + \varepsilon)$'s side untouched:

$$ri_1 = []@([]@(0@ε \oplus 1@b) \cdot []@(0@b \oplus 1@ε))$$

ri_1 's left operand is now nullable (via $0@ε$), so deriving with respect to **b** takes the *Seq* case's other branch: the left operand's own derivative continues on one side of a freshly introduced *Alt*, while *mkEpsBC* of that same left operand – $[0]$, the bit its nullable $0@ε$ disjunct contributes – is fused onto the right operand's derivative on the other:

$$ri_2 = []@([]@(1@([]@(0@b \oplus 1@ε)) \oplus [0,0]@ε))$$

(the leading 1 tags "derivative continued into the left operand" as the *Right* alternative of this new node, since unlike *inj* – which only discovers this tagging when reconstructing backward – *deriv_bc* must commit to it immediately as part of the forward pass itself; the second 0 in $[0,0]$ is $(b+\varepsilon)$'s own not-yet-derived left-disjunct tag, fused after ri_1 's $[0]$). ri_2 's top-level *Alt* has *both* disjuncts nullable here – $[0,0]@ε$ trivially, and $1@([]@(0@b \oplus 1@ε))$ as well, via its own nested right disjunct $1@ε$ – so *mkEpsBC*'s *Alt* rule, which (mirroring *mkEps*'s own *Left*-preference from Section 5.1) always takes the left disjunct when it is nullable, regardless of the right disjunct also qualifying, picks $1@(\dots)$ over the shorter $[0,0]@ε$:

$$mkEpsBC_{ri_2} = [] ++ mkEpsBC_{1@([]@(0@b \oplus 1@ε))} = 1 ++ ([] ++ mkEpsBC_{1@ε}) = 1 ++ [1] = [1,1]$$

the same left-preference that makes r_1 's plain construction choose *Right* at the top level in Section 5.1's worked example: both encode "prefer continuing the match already in progress over falling back to the shorter alternative," just applied at different points – there, during *inj*'s backward pass; here, directly inside *mkEpsBC*'s forward-accumulated bits. Decoding against the original r_1 :

$$decode_{r_1}([1,1]) = (Right(a,b), Right())$$

– the same tree Section 5.1's worked example reached via its explicit backward pass, this time read directly off the bits accumulated during the forward pass alone.

On simplification between steps. Section 5.1 noted that the plain construction's forward pass must *not* simplify between steps, since the backward pass needs to walk back

through the exact sequence of derivatives that produced it. The bit-coded construction has no such restriction, and `parse_bitcoded_recursive/parse_bitcoded_loop` (`src/parsers/deriv_bc/parse.rs`) do call `simp` after every step, exactly as this worked example's derivation does. This is safe here for the same reason `fuse` is: the information a backward pass would otherwise need to walk back for is already carried *on* the annotated expression itself, as bits attached to its nodes, so simplifying ri_i down to a smaller but bit-equivalent expression loses nothing *mkEpsBC* needs later – unlike the plain construction, where simplifying r_i would destroy the very structure *inject* needs to reconstruct against.

5.4 Simplification Rules

Section 3.2's worked example already showed plain derivatives growing an extra $+$ at every nullable-prefix step with nothing to bound that growth; the bit-coded construction inherits the same problem; [Sulzmann and Lu, 2014] give the concrete case $a^* \xrightarrow{a} \varepsilon a^* \xrightarrow{a} \phi a^* + \varepsilon a^* \xrightarrow{a} (\phi a^* + \varepsilon a^*) + (\phi a^* + \varepsilon a^*) \xrightarrow{a} \dots$, where every step after the first is language-equivalent to εa^* but syntactically doubles in size. Left unchecked, this defeats the whole point of the bit-coded construction's incremental, single-pass promise: an unboundedly growing ri makes each subsequent $\backslash_b$ step correspondingly more expensive.

A simplification function *simp*, applied to a fixpoint after every derivative step, keeps the annotated expression's size bounded. Its rules – Phi-propagation through *Seq/Alt*, epsilon-elimination ($(bs@(bs'@\varepsilon) ri) \rightarrow fuse(bs ++ bs', ri)$, folding the eliminated node's own bits forward rather than discarding them), and flattening/deduplicating nested alternations, keeping the first occurrence of a syntactic duplicate – are all instances of the familiar algebraic identities from Section 3.2 ($\emptyset \cdot r = \emptyset$, $\varepsilon \cdot r = r$, $\emptyset + r = r$, $r + r = r$), applied here to the annotated grammar and combined with an explicit *isPhi* test that recognizes "empty language" expressions built up from these rules rather than only the literal $\emptyset$ constructor. Applied to the worked example above, *simp* would collapse every step back down to (a bit-annotated form of) εa^* , exactly as ordinary `simplify` does for the un-annotated case.

Sulzmann and Lu claim, but do not fully prove, that the resulting construction runs in time linear in the input, on the grounds that each of $\backslash_b$, *simp*, *fuse*, *mkEpsBC*, and *isPhi* only ever recurses over sub-calls bounded by the (now-simplified) expression's size, and note explicitly that detailed space-complexity estimates are left as future work [Sulzmann and Lu, 2014]. This thesis does not attempt to close that gap either; Chapter 8's benchmarks measure the implementation's actual behavior empirically rather than establishing a tighter asymptotic bound than the reference itself provides.

5.5 Rust Implementation: The `deriv_std` Parser

`deriv_std` (`src/parsers/deriv_std/`) is the Rust realization of Section 5.1's direct construction. It exposes two entry points computing the identical result by two different means – `parse_recursive` and `parse_loop` – discussed together in Section 5.5.4 below, after the shared machinery both call into.

5.5.1 ParseTree and flatten

The `ParseTree` type and its `flatten` function were already introduced in Section 4.2 (`src/types/tree.rs`) – `Empty`, `Char`, `Pair`, `Left`, `Right`, `Star` corresponding one to one with `()`, a literal, a pair, the two tagged injections, and a Kleene-star list. Nothing about the type itself is specific to `deriv_std`: it is the one representation every parser in this and the following chapter ultimately produces, precisely so that the agreement tests of Chapter 8 can compare their outputs directly rather than through some intermediate translation.

5.5.2 mk_eps

`mk_eps` (`src/regex/mk_eps.rs`) transcribes *mkEps* case for case: `Eps` to `Empty`, `Star` to an empty `Vec`, `Seq` to a `Pair` of the two recursive results, and `Alt` to `Left/Right` depending on `nullable(r1)` exactly as the theory prescribes. Called on `Lit` or `Phi` – which `mkEps` has no case for, since neither is ever nullable – it panics rather than silently returning a meaningless value; every call site only ever reaches `mk_eps` once nullability of its argument has already been established, so the panic is an assertion of an invariant the surrounding code maintains, not a reachable runtime error. This function lives in `regex/`, flat and outside `deriv_std/` specifically, because Chapter 6's `pderiv_std` needs the identical function for the identical reason – both build a `ParseTree` directly rather than bit-codes, and both need the same "parse tree for a nullable expression matched against nothing" primitive to do it.

5.5.3 inject

`inject` (`src/parsers/deriv_std/inject.rs`) is the direct transcription of *inj*. Its `Seq` case is the one worth reading closely against the theory: when `r1` is not nullable, only a `Pair` is ever handed in, and injection recurses into its first component; when `r1` is nullable, the incoming value is either `Left(Pair(v1, v2))` (the "continue `r1`" branch the derivative rule produces) or `Right(v2)` (the "`r1` already matched empty" branch), and only the second case calls `mk_eps(r1)` to synthesize the otherwise-absent left component – exactly the three-way split of Section 5.1's $inj_{(r_1 r_2) \setminus x}$. Every case that receives a value shaped differently from what the corresponding derivative rule could have produced – a bare `Pair` where a nullable `r1` demands a tagged `Left/Right`, for instance – panics rather than guessing, since such a mismatch can only mean the caller passed a tree that did not actually come from `r`'s own derivative sequence.

5.5.4 parse_recursive vs. parse_loop

Both functions (`src/parsers/deriv_std/parse.rs`) compute the identical result – shown empirically, not just by construction (Chapter 8) – by two different means of implementing the same two-phase algorithm. `parse_recursive` recurses to the end of the input first, then calls `inject` on the way back up the call stack as the recursion unwinds – `deriv`'s "leading character removed" and `inject`'s "leading character re-inserted" mirror each other directly across the recursion, needing no explicit intermediate storage for the forward-pass sequence at all, since the call stack is that sequence. `parse_loop` makes the same sequence explicit as a `Vec<Regex>`, built by an ordinary forward `for` loop, then walks that vector backward with a second loop calling `inject` at each step. The two exist

side by side because they make different resource trade-offs unrelated to the algorithm itself: `parse_recursive`'s call stack depth grows with the input length, and can overflow it on sufficiently long input, where `parse_loop`'s explicit `Vec` instead grows on the heap – Chapter 8 measures exactly where that difference starts to matter.

5.6 Rust Implementation: The Bit-Coded Version

`deriv_bc (src/parsers/deriv_bc/)` realizes Section 5.3's construction.

5.6.1 ARegex – Bit-Annotated Regex

`ARegex (src/types/aregex.rs)` mirrors the annotated grammar *ri* directly: every variant except `Phi` carries its own `Vec<bool>` bit-prefix alongside the same structural payload `Regex` already carries (a `char` for `Lit`, boxed sub-expressions for the recursive cases). This is the one point in the whole implementation where a plain `Regex` is not enough: `deriv_bc`'s entire purpose is threading partial parse-tree information *through* the tree as it is derived, and that requires a place on every node to put it, not only at the leaves.

5.6.2 internalize/fuse

Both in `src/parsers/deriv_bc/internalize.rs`, `internalize` converts a plain `Regex` into a freshly bit-annotated `ARegex` with every list empty, except `Alt`: the left branch is fused with a leading `false` and the right with a leading `true` at construction time, exactly as the theory's *internalize* does. `fuse(bs, ri)` prepends `bs` onto `ri`'s own top-level annotation and is the one primitive the rest of this section's functions are all built from – including the degenerate case `fuse(bs, Phi) = Phi`, since there is no annotation slot on `Phi` to prepend anything to.

5.6.3 deriv_bc

`src/parsers/deriv_bc/deriv.rs` transcribes $\backslash_b$ case for case. Its `Alt` case keeps the outer bit-list on the resulting `Alt` node itself, rather than pushing it into either recursive call – a detail that looks like an arbitrary implementation choice here but turns out to matter a great deal once partial derivatives are introduced in Chapter 6, where the same choice, applied to a construction that no longer has a single combined node to attach bits to, is shown to cause genuine performance problems rather than a merely cosmetic difference. Its `Seq` case implements the nullable- r_1 branch's two strands exactly as written above – continue r_1 's own derivative, or jump into r_2 's with r_1 's empty-match bits fused on – and its `Star` case attaches the leading "one more iteration" bit via `fuse(&[false], ...)` before rewrapping the untouched star body. `deriv_bc` is called in a loop, once per input character, inside `parse_bitcoded/parse_bitcoded_loop (src/parsers/deriv_bc/parse.rs)`, with `simp` applied after every step – the Rust realization of Section 5.4's fixpoint simplification, in `src/parsers/deriv_bc/simplify.rs`, whose cases (Phi propagation, epsilon-elimination via `fuse`, and list-flattening/first-occurrence deduplication for nested alternations) match *simp*'s directly.

5.6.4 mk_eps_bc

`src/parsers/deriv_bc/mk_eps_bc.rs` extracts the accumulated bit-string from a nullable `ARegex`, structurally identical to `mk_eps` but returning bits instead of a tree: `Alt` prefers whichever branch is nullable (left checked first, matching `mk_eps`'s own left-preference), `Seq` concatenates both children's bits after its own prefix, and `Star` appends a final `true` marking "no further iterations" onto whatever bits the completed iterations already accumulated.

5.6.5 decode

`src/regex/decode.rs` turns the plain, *original* `Regex` plus a flat `Vec<bool>` bit-string back into an ordinary `ParseTree`: `Alt` consumes one leading bit to choose `Left/Right`, `Seq` decodes both children in sequence, and `Star` loops, consuming a `false` per iteration until a `true` ends it – the direct inverse of `encode`, which this implementation never needs to run itself, since nothing here starts from an already-built `ParseTree` needing to be re-encoded. `decode` takes only a plain `Regex`, not an `ARegex` – by the time it is called, the bit-string alone carries everything needed, decoded against the shape of the *un*-annotated original expression. This is exactly why `decode` lives in flat `regex/` rather than inside `deriv_bc/` specifically: Chapter 6's `pderiv_bc` produces the same `(Regex, bits)` pair by an entirely different route and needs the identical final step to turn it into a tree.

5.7 Worked Example: paper_r1 End to End

Section 4.2 introduced $r_1 = (a + ab)(b + \varepsilon)$ against `ab` (`paper_r1`) on paper, deriving by hand that the POSIX result is $v = (Right(a, b), Right())$. What follows is that same derivation at diagnostics level 3 – the full structural trace, not the summarized construction steps of level 2 – produced by running `deriv_std` and `deriv_bc` themselves (`cargo run - parse "(a|ab)(b|ε)" "ab" -diag 3`, and again with `-parser deriv_bc`) rather than reconstructed from the theory alone: every derivative rule application, every intermediate expression, and every nullability check this chapter's two constructions actually perform, character by character.

Success: `deriv_std`. The forward pass names each rule applied at each step – Section 5.1's own $\partial(r_1 \cdot r_2, x)$ case split, visibly taken once per character – followed by the backward pass (Section 5.5.2/5.5.3) naming each *mkEps/inject* call in turn:

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
=====
TIMING
=====
```

```

Parse time:          0.011ms
Expressions stored:  3 (r0 -> r1 -> r2)
-----=====
FORWARD PASS (Derivatives)
-----=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a') [nullable(r1)=false]
        = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true [OK] position 1 matched

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=true]
        = d(r1, 'b').r2 + d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
  nullable(r2) = true [OK] position 2 matched

-----=====
NULLABILITY CHECK
-----=====
nullable(r2) = true -> proceed to backward pass
-----=====
BACKWARD PASS (mkEps + inject)
-----=====
mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())

inject(r1, 'b', v2):
  inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)),
    'b', Left (Right Right (), Right ()))
  -> (Right (), b), Right ()
v1 = (Right (), b), Right ()

inject(r0, 'a', v1):
  inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
    Eps)), 'a', (Right (), b), Right ()))
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())

-----=====
RESULT
-----=====
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"

```

```
-----=====
END OF REPORT
-----=====
```

Every intermediate r_1/r_2 this report names is printed in full, not just referenced by number – **Expressions stored:** 3 ($r_0 \rightarrow r_1 \rightarrow r_2$) is exactly the derivative sequence Section 5.1 builds, and the backward pass’s own $v_2/v_1/v_0$ trace $mkEps(r_2)$ through two **inject** calls back to the final tree – character-for-character the v Section 4.2 derived on paper, as Theorem 1 (Section 5.2) guarantees it must be.

Success: **deriv_bc.** The same input against the same r_1 , through the bit-coded construction of Section 5.3: **internalize** annotating the regex once, then each **deriv_bc** + **simp** step shown against the derivative-on-bits rule it applies, then **mkEpsBC** extracting bits and **decode** turning them back into a tree:

```
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
```

```
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
```

```
-----=====
TIMING
-----=====
```

```
Parse time:          0.014ms
Derivative steps:    2
```

```
-----=====
INTERNALIZE
-----=====
```

```
internalize((a|ab)(b|epsilon))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b'))) .
      []@([false]@'b' (+) [true]@epsilon))
```

```
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
```

```
Step 1 (char: 'a', position 1)
  ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b'))) .
        []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'a'):
    rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
          = bs@(ri1\'a'.ri2)
  simp -> []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b'
    (+) [true]@epsilon))
  ri1 = []@([]@([false]@epsilon (+) [true]@'b') . []@([false]@'b'
    (+) [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating
```

```

Step 2 (char: 'b', position 2)
  ri1 = []@([]@([false]@epsilon (+) [true]@'b')) . []@([false]@'b'
    (+) [true]@epsilon))
  deriv_bc(ri1, 'b'):
    rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
      = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
  simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
    false]@epsilon)
  ri2 = []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
    false]@epsilon)
  nullable_bc(ri2) = true [OK] bits accumulating

-----
NULLABILITY CHECK
-----
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----
MKEEPSBC + DECODE
-----
mkEpsBC(ri2) = [11]

decode((a|ab)(b|epsilon), [11]):
  = (Right (a, b), Right ())
-----
RESULT
-----
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----
END OF REPORT
-----

```

Where `deriv_std`'s report names three plain `Regex` expressions and two `inject` calls, `deriv_bc`'s names two annotated `ri0 -> ri1 -> ri2` steps and a single `mkEpsBC/decode` pair – the same structural difference Sections 5.5 and 5.6 describe in Rust terms, now visible directly in each construction's own trace. The final bits `[11]` select *Right* at both alternation points – the second alternative of $a + ab$, then the second (empty) alternative of $b + \varepsilon$ – which `decode` reads back as the identical $(\textit{Right}(a, b), \textit{Right}())$ `deriv_std`'s separate backward pass also produced.

Failure: `deriv_std`. Running the same r_1 against `b` instead – a string not in $L(r_1)$, since every derivation of r_1 must begin by matching `a`:

```

-----
REGEX ENGINE DEBUG REPORT
-----
Mode:                POSIX Standard (recursive parser)

```

```

Regex:          (a|ab)(b|epsilon)
Input:          "b"
Result:         NO MATCH
Policy:         POSIX
-----=====
TIMING
-----=====
Parse time:      0.006ms
Expressions stored: 2 (r0 -> r1)
-----=====
FORWARD PASS (Derivatives)
-----=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

-----=====
NULLABILITY CHECK
-----=====
nullable(r1) = false -> no parse tree exists for full input
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
  b
  ^
-----=====
END OF REPORT
-----=====

```

The single derivative step taken, $\partial(r_1 \cdot r_2, b)$ with r_1 not nullable, derives r_1 down to a Seq of two Phis – not nullable – so the report moves directly from the nullability check to partial recovery and an error summary, never reaching a backward pass at all.

Failure: deriv_bc. The bit-coded construction on the identical input:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====

```

```

Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|epsilon)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.006ms
Derivative steps:    1
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|epsilon))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'b', position 1)
  ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
  deriv_bc(ri0, 'b'):
    rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
      = bs@(ri1\'b'.ri2)
  simp -> phi
  ri1 = phi
  nullable_bc(ri1) = false [FAIL]
-----=====
NULLABILITY CHECK
-----=====
nullable_bc(ri1) = false -> no full parse tree exists
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
  b
  ^
-----=====
END OF REPORT

```

-----=====

`simp` collapses the single derivative step all the way to `phi` rather than stopping at an unreduced `Seq` of `Phis` the way `deriv_std`'s unsimplified report does – consistent with Section 5.4's own $\phi \cdot r \equiv \phi$ rule firing immediately – but both constructions report the identical position, character, and expected-set in their final error summary, since the diagnostics layer's expected-set (Chapter 9's Section 9.3.3) is read directly off r_1 's first literal set at the point of failure, independently of which construction, or how much simplification, produced it.

Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative, both computing the identical POSIX $\geq_r$ -maximal tree of Section 4.3.3. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout – but, as Section 6.1 shows first, repeating the construction with a different derivative operator does not repeat its disambiguation guarantee. Neither flops14-extended [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann] defines a POSIX partial-derivative parser at all – the paper’s bit-coded construction (Chapter 5) is Brzozowski-derivative-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a dedup rule this chapter shows to be fundamentally Greedy rather than POSIX. This chapter therefore builds two *Greedy* parsers, `pderiv_bc` (Section 6.3, following `pDerivBC` directly) and `pderiv_std` (Section 6.4, this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`) – both verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8.

6.1 Extending Derivatives to Partial Derivatives

Section 3.3 already defined $pd(r, x)$ as a *set* of residual expressions rather than one combined expression, and noted the NFA reading: each element of $pd(r, x)$ is one state of an implicit NFA the original r denotes, alive after reading x . Extending this to parsing means pairing each surviving element of that set with enough information to reconstruct a parse tree once a particular strand turns out to be the winning one at the end of the input – concretely, either an accumulated bit-string per strand (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per strand (Section 6.4). Either way, the algorithm shape is the same: maintain a *frontier* of `(residual, extra)` pairs, one per live strand, replace it at each character with the union of pd applied to every current strand (each contributing its own possibly-empty set of successor strands), and at the end of the input select the first strand in the frontier whose residual is nullable.

Why the result is Greedy, not POSIX. This last step – “the *first* nullable strand in the frontier” – is where this construction’s disambiguation policy is actually decided, and it is decided implicitly, as a consequence of how the frontier’s growth is kept finite rather than by any explicit ordering rule analogous to Section 4.3.3’s (A1)-(A4)/(K1)-(K4). Antimirov’s finiteness theorem (Section 3.3) guarantees the set of *distinct* partial derivatives is bounded, but nothing about the extended, tree-or-bit-carrying construction guarantees strands stay distinct as they accumulate – two different derivation paths can easily reach the identical residual *shape* by two different routes, each carrying its own bits

or injection. Kept without pruning, this set would multiply extra copies of an already-fully-explored residual instead of the bounded, syntactic set *pd* itself guarantees; pruned by comparing the accumulated bits or trees for POSIX-style "longest match wins" the way Section 4.3.3's ordering does, the resulting comparison would need to look inside possibly-incomplete, still-growing accumulated state rather than a finished `ParseTree`, which the ordering relation of Definition 4.3.3 is not defined over. The Haskell reference's actual choice – `nub2`, deduplicating purely on residual *shape* and keeping whichever strand reached that shape *first* – sidesteps this entirely by discarding the comparison altogether: once a residual shape has been claimed by some strand, every later strand reaching the identical shape is simply dropped, regardless of what either one's accumulated bits or tree would eventually have produced. "First" here means first in the list-ordered traversal *pd*'s own case structure produces, which explores the left operand of $+$ before the right at every step – so "first claim wins" is exactly Greedy's leftmost-commits-immediately rule (Section 4.3.1), not POSIX's longest-wins-with-left-as-tiebreak rule. This reading is this thesis's own account of why the construction lands on Greedy rather than POSIX; neither primary source states it in these terms; both parsers built in this chapter are verified empirically in Chapter 8 to compute exactly the Greedy answer wherever it is known to differ from POSIX (the `paper_r1`/`paper_r2` examples of Section 4.3), which is the strongest evidence available for it in the absence of a reference construction to check against directly.

6.2 Bit-Coded Variant

Sulzmann's teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis's `pderiv_bc` follows directly [Sulzmann]. It extends *pd* with per-strand bits the same way Section 5.3's $\backslash_b$ extends the Brzowski derivative – but, notably, keeps those bits *external* to the regular-expression tree, as a plain list carried alongside a plain, un-annotated *r*, rather than folding them into an `ARegex`-shaped tree the way $\backslash_b$ does:

$$\begin{aligned}
pDerivBC(x, \varepsilon) &= \{\} \\
pDerivBC(x, y) &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} \\
pDerivBC(x, r + s) &= nub2(\{(r', 0:bs) \mid (r', bs) \in pDerivBC(x, r)\} \cup \{(s', 1:bs) \mid (s', bs) \in pDerivBC(x, s)\}) \\
pDerivBC(x, rs) &= \begin{cases} nub2(\{(rs', bs) \mid (r', bs) \in pDerivBC(x, r), rs' = r's\} \\ \quad \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in pDerivBC(x, s)\}) & n(r) \\ \{(smartC(r', s), bs) \mid (r', bs) \in pDerivBC(x, r)\} & \text{otherwise} \end{cases} \\
pDerivBC(x, r^*) &= nub2(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in pDerivBC(x, r)\})
\end{aligned}$$

where $mkEpsBC_r$ is the plain-*r* analogue of Section 5.6.4's $mkEpsBC$ (identical structure, walking an un-annotated tree instead of an `ARegex`), and `nub2` deduplicates a list of (`residual`, `bits`) pairs purely by comparing the residual, keeping the first occurrence. The concatenation case's first strand deliberately keeps *r'* wrapped in a literal, non-smart-constructed *r's* rather than $smartC$ – an asymmetry present in the reference itself, preserved here rather than "corrected", since correctness of this construction is judged against literal agreement with that reference (Chapter 8), not against an independently-derived ideal form.

6.3 Rust Implementation: The pderiv_bc Parser

`src/parsers/pderiv_bc/pderiv.rs` transcribes *pDerivBC* case for case, as `Vec<(Regex, Vec<bool>)>` rather than a set (order matters here, encoding Greedy priority directly in list position, so a `HashSet` would lose exactly the information the construction depends on). `mk_eps_bits` implements *mkEpsBC_r*; `nub2` dedups on the residual `Regex` alone via a `HashSet<Regex>` used only to test membership, keeping the first-seen pair for each distinct residual and discarding the rest. `pderiv_bc`'s own `Seq` case matches the reference literally, down to the same asymmetric use of plain `Regex::seq` (not `smart_seq`) for its first strand.

`step_frontier/select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the frontier-driving loop of Section 6.1: at each character, `step_frontier` replaces every `(residual, bits)` pair in the current frontier with the union of `pderiv_bc` applied to it, concatenating each result's bits onto that strand's already-accumulated prefix; at the end of the input, `select_bits` scans the frontier in list order and returns the first nullable strand's `bits ++ mk_eps_bits(residual)`. `parse_pderiv_bc` drives this loop to completion and passes the winning bit-string, together with the *original* `Regex` (never the derived residual), to `decode` (Section 5.6.5) – the same shared decoder Chapter 5's `deriv_bc` uses, since both constructions end up at the identical `(Regex, bits)` shape by the time decoding is needed, whatever route each took to get there.

Worked example. Run Section 5.3's $r_1 = (a+ab)(b+\varepsilon)$ against `ab` through `pderiv_bc` instead of `deriv_bc` – same regex, same input, different parser, to see Greedy and POSIX visibly disagree on the same worked example rather than only being told they can. (Again, this is exactly `cargo run -p parse "(a|ab)(b|ε)" "ab" -parser pderiv_bc -diag 3`'s output.) The frontier starts as a single strand, the whole regex with no bits yet:

$$[(r_1, [])]$$

Deriving with respect to `a`: r_1 's left *Seq* operand is not nullable, so `pDerivBC`'s *Seq* case applies uniformly to the single strand, producing one residual per partial derivative of $(a+ab)$ with respect to `a` – two strands where the plain bit-coded construction kept one:

$$[(b+\varepsilon, [0]), (b \cdot (b+\varepsilon), [1])]$$

the first strand from a 's own derivative (ε , smart-concatenated away against $(b+\varepsilon)$), tagged 0 for having come from $(a+ab)$'s left disjunct; the second from ab 's derivative (b , concatenated onto $(b+\varepsilon)$), tagged 1 for the right disjunct. Deriving with respect to `b` steps *both* strands independently: strand 0's $(b+\varepsilon)$ splits again into two partial derivatives of its own (ε from the left disjunct, $\emptyset$ from the right, `nub2`-deduplicated and the $\emptyset$ one dropped), while strand 1's $b \cdot (b+\varepsilon)$ takes the *Seq* case's not-nullable branch and derives only its left factor:

$$[(\varepsilon, [0,0]), (b+\varepsilon, [1])]$$

Selection scans this final frontier in list order and takes the *first* nullable strand – strand 0, $(\varepsilon, [0,0])$, is checked and accepted before strand 1 is even inspected, regardless of strand 1 also being nullable. This is Greedy's leftmost-commits rule made mechanical: strand 0 is exactly the "matched `a` then `b` separately" reading, the first alternative `pDerivBC` generated back at the first derivative step, and it wins here purely because it appears first

in the frontier – not because any length comparison favors it. Decoding its accumulated bits:

$$\text{decode}_{r_1}([0, 0]) = (\text{Left } a, \text{Left } b)$$

the *shorter* $a + b$ reading – the opposite answer from Section 5.3’s $(\text{Right}(a, b), \text{Right}())$ on the identical regex and input. Nothing about r_1 or “ab” changed between the two traces; only the disambiguation policy governing which nullable strand/branch wins did, exactly the distinction Section 6.1 argues is built into nub2’s shape-only, first-occurrence-wins deduplication.

6.4 Rust Implementation: The pderiv_std Parser

6.5 Worked Example: paper_r1 End to End

Section 4.3 constructed, by hand, the *second* valid derivation of $r_1 = (a + ab)(b + \varepsilon)$ against ab (paper_r1) that Section 5.7 deliberately left aside: $v' = (\text{Left } a, \text{Left } b)$, matching the *first* alternative of $a + ab$ against a alone rather than the second alternative against ab in one step. This is the Greedy result – leftmost-alternative priority, decided locally at $a + ab$ without looking ahead at what $b + \varepsilon$ could still consume – and it is exactly what pderiv_std and pderiv_bc compute, not $v = (\text{Right}(a, b), \text{Right}())$ from Section 5.7. What follows is the full diagnostics level 3 trace for both – every partial derivative computed at every frontier step, not the summarized residual list level 2 prints – run against the identical r_1/ab (cargo run - parse "(a|ab)(b|ε)" "ab" -parser pderiv_std -diag 3, and again with -parser pderiv_bc): the frontier construction this chapter developed, actually computing the *other* tree Section 4.3 worked out by hand, and a concrete demonstration that this chapter’s parsers and Chapter 5’s are not interchangeable substitutes for the same problem: same input, same regular expression, two different, individually correct answers.

Success: pderiv_std. Each step shows every residual in the frontier *before* the step, the rule (Section 6.4’s pderiv_tree, one case per residual depending on whether its own head is nullable) applied to each, and the resulting frontier *after*:

```

=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
=====
TIMING
=====
Parse time:          0.018ms
Steps:               2
=====
INITIAL FRONTIER

```

```

-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
        = [(r1'.r2, inj: continue r1) | (r1',_) <-
          pderiv_tree(c,r1)]
      -> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
        printable)
      -> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection
        closure, not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree
    available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
        = nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
          pderiv_tree(c,r1)]
          ++ [(r2', inj: mk_eps(r1) paired with r2's
            result) | (r2',_) <- pderiv_tree(c,r2)]
      -> Eps (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
        = [(r1'.r2, inj: continue r1) | (r1',_) <-
          pderiv_tree(c,r1)]
      -> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
        printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree
    available

```

```

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to
  injection (already applied by select)
=====
RESULT
=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

Step 1's frontier already shows the fork this chapter's theory predicts: residual [0], `Seq(Eps, Alt(Lit('b'), Eps))`, is the strand that consumed `a` via `a+ab`'s *first* alternative; residual [1], nested one step deeper, is the strand that instead committed to `a+ab`'s *second* alternative and still owes a `b` before `b + ε` is even reached. `nub_tree` keeps both alive (they are structurally distinct residuals), and at step 2 the first-listed, first-priority residual is the one that reaches `Eps` – nullable – first: Greedy's leftmost-alternative priority, operating directly on frontier order rather than on any global comparison between the two candidate trees.

Success: `pderiv_bc`. The bit-coded sibling (Section 6.3) reaches the same tree via `nub2-deduplicated (Regex, bits)` pairs instead of `(Regex, Inj)` closures, each frontier entry carrying its accumulated bits alongside its residual:

```

=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|epsilon)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
=====
TIMING
=====
Parse time:          0.036ms
Steps:               2
=====
INITIAL FRONTIER
=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
=====
FORWARD PASS (pDerivBC per residual, frontier flattened)
=====

```

```

Step 1 (char: 'a', position 1)
frontier before (1 residual(s)):
  [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
    Eps))  bits-so-far = []
pDerivBC('a', .) applied to each:
  from [0]:
    rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [0])
    -> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
frontier after (2 residual(s)):
  [0] Alt(Lit('b'), Eps)  bits = [0]
  [1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

```

```

Step 2 (char: 'b', position 2)
frontier before (2 residual(s)):
  [0] Alt(Lit('b'), Eps)  bits-so-far = [0]
  [1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits-so-far = [1]
pDerivBC('b', .) applied to each:
  from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
          tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
  from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
  [0] Eps  bits = [00]
  [1] Alt(Lit('b'), Eps)  bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

```

```

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to
  mkEpsBC + decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|epsilon), [00]):
  = (Left a, Left b)
=====
RESULT
=====

```

```

Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"

```

```

-----=====
END OF REPORT
-----=====

```

Bits [00] select *Left* at both alternation points – exactly v' 's shape – and the winning residual at each step is identical to `pderiv_std`'s first-listed one above, consistent with `pderiv_tree` being built to mirror `pderiv_bc`'s case structure one to one (Section 6.4).

Failure: `pderiv_std`. Against `b` instead of `ab`: the first partial derivative of r_1 by `b` produces no residual at all, since neither alternative of $a+ab$ begins with `b`, so the frontier is empty after step 1 and no selection ever runs:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|epsilon)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
-----=====
TIMING
-----=====
Parse time:          0.005ms
Steps:               1
-----=====
INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
        = [(r1'.r2, inj: continue r1) | (r1',_) <-
          pderiv_tree(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

```

```

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = false -> no full parse
  tree exists
-----=====
PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====
ERROR SUMMARY
-----=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
-----=====
END OF REPORT
-----=====

```

Failure: pderiv_bc. The bit-coded construction on the identical input, reaching the same empty frontier by the same rule:

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|epsilon)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
-----=====
TIMING
-----=====
Parse time:          0.004ms
Steps:               1
-----=====
INITIAL FRONTIER
-----=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
-----=====
FORWARD PASS (pDerivBC per residual, frontier flattened)
-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
    [0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'),
      Eps))  bits-so-far = []

```

```

pDerivBC('b', .) applied to each:
  from [0]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> []
frontier after (0 residual(s)):
no nullable residual in this frontier [FAIL]

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = false -> no full parse
  tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces – Greedy versus POSIX – but agree exactly on the boolean question of membership and on the diagnostics reported for a non-match, since neither disambiguation policy nor bit-versus-closure tree construction is ever reached until after the underlying derivative/partial-derivative walk has already found at least one live residual.

Frontend: Parsing External Regular Expression Syntax

Every example so far in this thesis has been written directly in **Regex** terms – the six-constructor core algebra of Section 2.3, exactly what `deriv/pderiv` operate on. No real regular-expression user writes that way: a Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string* – “[a-c]+\d{2}”, not `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the **Regex** every parser in Chapters 5 and 6 already understands. It is modeled throughout on `Text.Regex.PDeriv`’s own `ExtPattern/Parse/Translate` modules from `luzhuomi/regex-pderiv`, Kenny Lu’s own reference implementation of [Sulzmann and Lu \[2012\]](#)’s partial-derivative submatching – a second, independent reference alongside [Sulzmann and Lu \[2014\]](#)’s own teaching-page material (Section 9.2.1), used here specifically because the `flops14`-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor **Regex** algebra this thesis’s core inherits, with no PCRE-facing frontend of its own.

Section 8.3’s real-world corpus depends on everything described here (surface-syntax parsing, desugaring, and the substring-search padding scheme) to turn raw `Suricata/SpamAssassin/RegexLib` rule text into **Regex** values in the first place.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into **Regex**:

$$\text{pattern string} \xrightarrow{\text{parse_ext_pattern}} \text{ExtPat} \xrightarrow{\text{translate}} \text{Regex}$$

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy: bool` flag on every quantifier (`a*` vs. `a*?`) – everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. **Regex** (Section 2.3) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, no groups, no bounds, no anchors, no laziness flag. Anything `ExtPat` can say that **Regex** cannot represent is desugared away by `translate` (Section 7.3) or, for `^/$` specifically, erased into `Eps` and handled one level up instead (Section 7.4).

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from parser to **Regex** in one pass, is what makes `translate` independently testable: `translate.rs`’s

own unit tests (below) construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on `parse.rs`'s recursive-descent logic at all. It is the same separation-of-concerns argument Section 5.5.1 makes for keeping `ParseTree` construction and `flatten/decode` as distinct steps rather than one fused pass – here applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after `Text.Regex.PDeriv.Parse`'s own grammar – `p_ere/p_branch/p_exp/p_atom/p_group/p_charclass/p_enum/p_bound` become `parse_or/parse_concat/parse_postfixed/parse_atom/parse_group/parse_charclass/parse_class_enum/parse_bound` one for one, each handling exactly the surface-syntax layer its reference counterpart does (alternation, then concatenation, then postfix quantifiers, then a single atom). Four deliberate departures:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference’s `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/ s/S` into character classes (Section 7.3’s `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** `\1–\9` and `(?=/(?!/(?<=/(?<!` produce a descriptive parse error rather than the reference’s literal-digit or 3-digit octal-ASCII escape reading – both describe languages that are not regular in the formal sense this thesis’s entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3’s extraction pipeline depends on this refusal being loud and categorized, not silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, dropped to their unnamed form during `translate` – a name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but rejecting a construct this common in real Suricata/RegexLib rules purely because the reference parser predates it would reject working patterns for no benefit.
- **\xHH hex escapes are accepted**, another addition absent from the reference, for the same reason.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`'s `greedy`: `bool` flag – correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation, and concatenation otherwise follow the reference's precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom (literal, escape, character class, group, or anchor).

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	translate(r) (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(translate(r), Eps)
Plus(r)	Seq(translate(r), Star(translate(r)))
Star(r)	Star(translate(r))
Carat, Dollar	Eps (see Section 7.4)
Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Bound(r, lo, hi)	unrolled exact/optional repetition, below

Character classes are ordinary alternation, not a primitive. Any/NoneOf/Dot/class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain Alt chain of Lit nodes. `alphabet.rs` supplies the underlying sets: `alphabet()` (98 characters – printable ASCII plus tab, newline, carriage return), `digit_chars/word_chars/ space_chars` for `\d/\w/ \s`, and `complement` (alphabet minus a given set) for their negated/uppercase counterparts and `[^...]`. There is no dedicated "character class" Regex variant anywhere in this project (Section 9.5.2 returns to this as an unexploited optimization) – every one of the four parsers only ever needs to understand Regex's original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way Alt the parser walks one branch at a time on every character.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times (`repeat_exact`, a left-folded Seq chain, Eps for `lo == 0`), then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies (`optional_extra`, each layer `Alt(Eps, Seq(r, ...))`) for a bounded one (`r{2,4}`). Unlike every other desugaring in this table, this one can make a pattern arbitrarily large from a short piece of source text – `r{1586,}` (Section 8.3's own example of a rejected real-world bound) would otherwise unroll into 1586 nested Seq nodes before the Star even starts. `MAX_BOUND` (1000 repetitions) is this project's own safety limit, not something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized Regex.

Greedy/lazy quantifier syntax is parsed but not honored. Every Opt/Plus/Star/Bound match arm above ignores the `greedy` flag `parse_bound` (Section 7.2) elaborated that Regex has no per-operator laziness concept at all, since POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy (`deriv_std/deriv_bc` versus `pderiv_std/pderiv_bc`, Section 4.3), not a per-quantifier one. `a*` and `a*?` produce byte-identical Regex values.. A pattern mixing lazy and greedy quantifiers, expecting per-operator control the way PCRE engines provide it, will therefore parse successfully but not behave as its author likely intended – a genuine expressiveness gap between the surface syntax this frontend accepts and what Regex can actually distinguish, noted here rather than left to be discovered.

7.4 Substring Search

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable" – there is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither [Sulzmann and Lu \[2014\]](#) nor Sulzmann’s teaching-page material defines one: POSIX/Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns – a Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern – so `frontend/mod.rs` fakes substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()` (`Star` over a 98-way `Alt` of the full alphabet, "any run of characters at all") on whichever side lacks an explicit anchor:

<code>^abc</code>	$\rightarrow abc \cdot \Sigma^*$	(leading anchor: pad the end only)
<code>abc\$</code>	$\rightarrow \Sigma^* \cdot abc$	(trailing anchor: pad the start only)
<code>abc</code>	$\rightarrow \Sigma^* \cdot abc \cdot \Sigma^*$	(no anchor: pad both sides)
<code>^abc\$</code>	$\rightarrow abc$	(fully anchored: no padding)

`starts_with_carat/ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat/Group` to find a leading `^` or trailing `$` even when nested, before `Carat/Dollar` are themselves translated away to `Eps` – their only real effect, under the search-oriented entry points `parse_dataset_pattern/parse_pcre_rule` (as against `parse_pattern`’s plain full-string equality, used everywhere else in this thesis’s own examples), is deciding which side of this padding is skipped.

Edge case. An unanchored pattern already nullable on empty input, `"a*"` being the simplest example, gets padded on both sides regardless of its own content – and the padding’s own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics ("some substring matches", and the empty substring of anything always does).

Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the Greedy $\geq_r$ -ordered parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other* – two POSIX, two Greedy, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness, Greedy/POSIX agreement on membership, the tree-vs-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section 4.3.1’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. This makes it a natural comparison point for this thesis’s Greedy parsers (`pderiv_std/pderiv_bc`) specifically, on both performance and, since both claim the same disambiguation policy,

parse-tree agreement – an even more direct check than the membership-only agreement this thesis’s own Greedy/POSIX comparison already establishes (Section 8.4).

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological $(a^*)^*$ -style expressions of Section 3.1 – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS"`; option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.3 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsupported `(?...)` group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns – a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern – so they are parsed for substring search via `frontend::parse_dataset_pattern/parse_pcre_rule` rather than the full-string-equality `parse_pattern` used elsewhere in this thesis’s own examples. Section 7.4 covers the mechanism in full – an unanchored core pattern is padded with an ordinary `Star` over a 98-way alternation on whichever side lacks an explicit `^/$` anchor, with no special status in the parser layer – and the one consequence worth restating here: real dataset patterns, run unanchored the way they have to be, are measurably bigger and expose scaling behavior the small synthetic patterns of Section 8.5.4 do not by themselves.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from Greedy on this pattern the way `paper_r1/paper_r2` do" – unlike Mamouras et al.’s own disambiguation-robustness analysis [Mamouras et al.,

2024], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible POSIX/Greedy divergence on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 9.5.2).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu’s paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 4.3.3: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 4.3.3; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the Greedy tree shape of Section 4.3.1 *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes, confirming this implementation reproduces both documented policies exactly on the paper’s own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.5.4), including `recursive_traced_and_loop_traced_agree_on_all_paper_c` comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever POSIX and Greedy are not known to diverge, and the two POSIX parsers (`deriv_std`, `deriv_bc`) and two Greedy parsers (`pderiv_std`, `pderiv_bc`) each agree exactly with their own policy-mate on tree shape – exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific

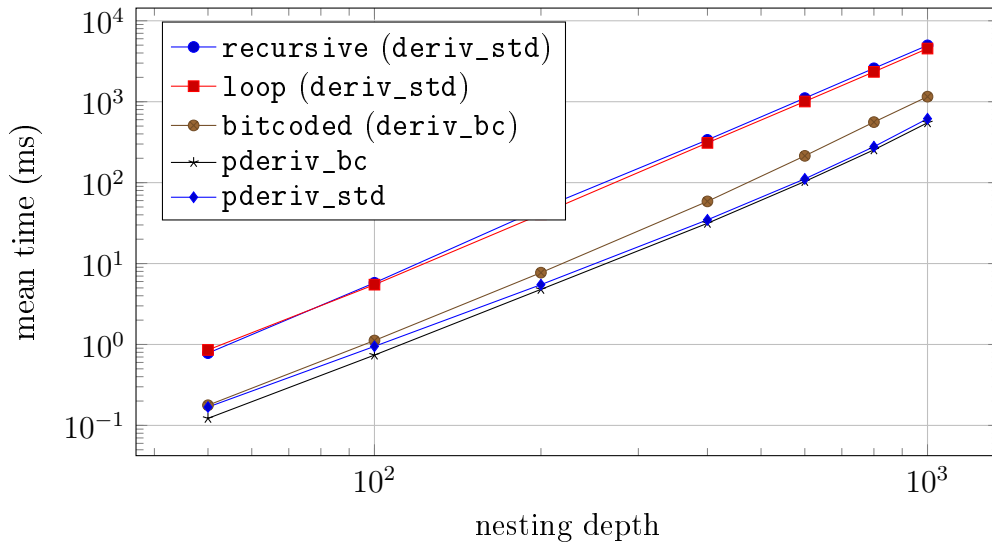
pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

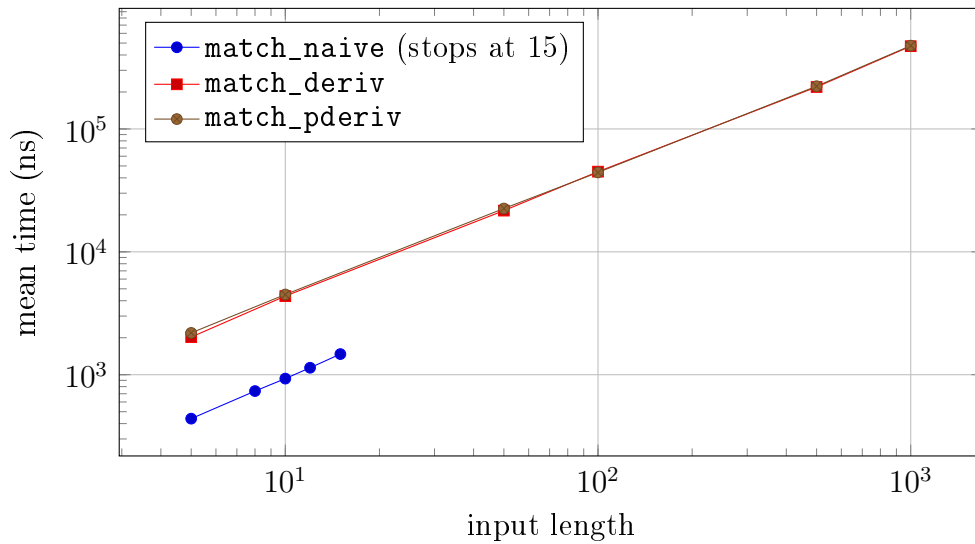
8.5.1 Stack Depth: Recursive vs. Iterative Construction

8.5.2 Heap Allocation: deriv_std vs. deriv_bc

8.5.3 Derivative-Based vs. Partial-Derivative-Based



8.5.4 Pathological Inputs



8.5.5 Benchmark Results

Parser	$n=5$	$n=10$	$n=15$	$n=18$
recursive (deriv_std)	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
bitcoded (deriv_bc)	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

Parser	Mean time / sweep
deriv_std_rec	1.311 s
deriv_std_loop	1.091 s
deriv_bc	643.5 ms
pderiv_bc	159.2 ms
pderiv_std	158.6 ms

Parser	Mean time
deriv_std_rec	1.000 s
deriv_bc	81.1 ms

8.6 Comparison with Existing Engines

8.6.1 Performance

Engine	Mean time / sweep	vs. regex
deriv_std_rec (POSIX, recursive)	1.503 s	$\times 1,415,254$
deriv_std_loop (POSIX, loop)	1.218 s	$\times 1,146,893$
deriv_bc (POSIX, bit-coded)	690.74 ms	$\times 650,417$
pderiv_bc (Greedy, bit-coded)	171.85 ms	$\times 161,815$
pderiv_std (Greedy, tree-injection)	173.28 ms	$\times 163,167$
regex (Rust crate)	1.06 μ s	$\times 1$
re2_perl (leftmost-first)	6.61 μ s	$\times 6.2$
re2_posix (leftmost-longest)	2.75 μ s	$\times 2.6$

Engine	Mean time / sweep	vs. regex
match_deriv	273.07 ms	$\times 259,815$
match_pderiv	134.95 ms	$\times 128,397$
regex (Rust crate)	1.05 μ s	$\times 1$
re2_perl (leftmost-first)	6.63 μ s	$\times 6.3$
re2_posix (leftmost-longest)	2.72 μ s	$\times 2.6$

8.6.2 Standardized Benchmarking via rebar

Discussion

9.1 Discussion of Evaluation Results

9.2 Rust versus Haskell

9.2.1 Language Characteristics of the Reference Implementation

The reference material this thesis translates from – both flops14-extended’s own presentation [Sulzmann and Lu, 2014] and Sulzmann’s fuller teaching-page Haskell transcript [Sulzmann] – leans on several Haskell features with no direct Rust counterpart, each already surfaced individually at the point in this thesis where it first mattered; this section collects them.

Algebraic data types with blanket deriving. The teaching page’s own `regex` type,

```
data R = Phi | Eps | L Char | Seq R R | Choice R R | Star R
    deriving (Show, Eq, Ord)
```

derives structural equality, ordering, *and* printing in one clause, whether or not a given piece of code actually uses all three. Section 2.3’s `Regex` derives only `Debug`, `Clone`, `PartialEq`, `Eq`, and `Hash` – no `Ord` – because nothing in this thesis’s Rust port ever compares two `Regex` values for ordering; Rust’s per-trait, opt-in `derive` makes that omission a visible, deliberate fact about the codebase, where the reference’s blanket `deriving` leaves it implicit.

GADT syntax used without a type index. The reference’s parse tree type is written as a GADT,

```
data U where
  Nil :: U
  Empty :: U
  Letter :: Char -> U
  LeftU :: U -> U
  RightU :: U -> U
  Pair :: (U,U) -> U
  List :: [U] -> U
```

but no constructor’s return type is ever refined to anything other than plain `U` – this is an ordinary sum type written in GADT syntax, not a use of GADTs’ actual type-level

power. Section 5.5.1’s `ParseTree` enum needs nothing beyond an ordinary Rust `enum` to match it.

9.2.2 What the Reference Provides vs. What It Leaves Open

9.2.3 Translation Strategy

9.2.4 Translation Insights

9.3 Mismatch Analysis via Derivative Tracing

9.3.1 The Gap

9.3.2 Deriving Diagnostics for Free from the Algorithm Itself

9.3.3 Design

`DiagLevel` (`src/diagnostics/mod.rs`) has four values – `Off` (0), `Basic` (1, `regex/input/-match/tree/error caret`), `Verbose` (2, `Basic` plus timing, expression count, and construction steps), and `Debug` (3, the full derivation trace, written to a file if `REGEX_DIAG_REPORT` is set) – selected via the CLI’s `-diag` flag or the `REGEX_DIAG` environment variable, and applies only to `parse`, not `match`: a Boolean matcher has no parse tree, and hence nothing at levels 1–3 to report beyond what the trace itself would show, which level 3 already provides directly. Each of the four parser variants renders its own trace independently rather than sharing one generic renderer – `diag_2/{deriv_std,deriv_bc,pderiv_std,pderiv_bc}.rs` and the analogous four files under `diag_3/` – since what is worth showing at each level differs by construction (a bit-coded parser’s trace is bit-strings and `ARegex` states; the tree-injection parsers’ traces are `Regex` derivative sequences and, at level 3, the backward injection steps that reconstruct a tree from them). `docs/DIAG.md` documents concrete output at every level, for every parser, on both successful and failing matches.

9.3.4 Scope and Limits

9.4 Optimization Challenges and Insights

9.5 Future Work

9.5.1 Paper-Suggested Extensions

Tagged NFA for Submatching

Space-Efficient Parsing

DFA/NFA Hybrid Abstraction

9.5.2 Further Performance Optimizations

Character Classes in Place of Literal Alternation

Structural Sharing via Rc

Memoizing `deriv/nullable`

Benchmarking the Matcher, Not the Parser, for Membership

9.5.3 A Full Mismatch Diagnostics System

Related Work and Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Oren Ben-Kiki, Clark Evans, and Ingy döt Net. YAML ain’t markup language (YAML) version 1.1. URL <https://yaml.org/spec/1.1/>.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- HitchDev. The norway problem: Why StrictYAML refuses to do implicit typing (and so should you). URL <https://hitchdev.com/strictyaml/why/implicit-typing-removed/>.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI), 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

The Flex Project. Patterns. URL <https://westes.github.io/flex/manual/Patterns.html>.

The Rust Project Developers. Using `Box<T>` to point to data on the heap, a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>.

The Rust Project Developers. Derivable traits, b. URL <https://doc.rust-lang.org/book/appendix-03-derivable-traits.html>.

The Rust Project Developers. Enums and pattern matching, c. URL <https://doc.rust-lang.org/book/ch06-00-enums.html>.

The Rust Project Developers. The `match` control flow construct, d. URL <https://doc.rust-lang.org/stable/book/ch06-02-match.html>.

The Rust Project Developers. `char` primitive type, e. URL <https://doc.rust-lang.org/std/primitive.char.html>.

The Rust Project Developers. `std::hash`, f. URL <https://doc.rust-lang.org/std/hash/index.html>.

The Rust Project Developers. `HashMap` in `std::collections`, g. URL <https://doc.rust-lang.org/std/collections/struct.HashMap.html>.

The Rust Project Developers. Enumerated types, h. URL <https://doc.rust-lang.org/reference/types/enum.html>.

The Rust Project Developers. Match expressions, i. URL <https://doc.rust-lang.org/reference/expressions/match-expr.html>.

The Rust Project Developers. Types, j. URL <https://doc.rust-lang.org/reference/types.html>.

Code Listings

Full Benchmark Results

Test Case Catalogue

CLI and ENV Reference